

دانشکده فنی و مهندسی ـ کامپیوتر

کارشناسی ناپیوسته نرم افزار

طراحی حرفه ای پنل ادمین سایت مارکتینگ

دانشجو : محمد پارسا عامری طورزنی

استاد راهنما : سرکار خانم دکتر شیرین عباسی

تابستان ۱۴۰۶-۱۴۰۵

به نام بی نام او

چکیده

این پروژه به توسعه و افزودن سه ماژول اصلی — سیستم مدیریت محتوای بلاگ، سامانه‌ی تحلیل رفتار کاربر، و فرم تماس متصل به صندوق پیام مدیریتی — به یک وب‌سایت شرکتی موجود مبتنی بر React می‌پردازد. پیش از این پروژه، وب‌سایت فاقد کانال انتشار محتوا، فاقد هرگونه داده درباره‌ی رفتار بازدیدکنندگان، و فاقد مکانیزم ذخیره‌سازی پیام‌های ارسالی کاربران بود.

برای پیاده‌سازی، به‌جای معماری سنتی سه‌لایه (فرانت‌اند، بک‌اند، پایگاه‌داده)، از رویکرد Backend-as-a-Service با استفاده از پلتفرم Supabase بهره گرفته شد؛ رویکردی که در آن فرانت‌اند React مستقیماً از طریق یک REST API خودکار (PostgREST) با یک پایگاه‌داده‌ی PostgreSQL ارتباط برقرار می‌کند، بدون نیاز به توسعه و نگه‌داری یک سرور بک‌اند اختصاصی. کنترل دسترسی میان بازدیدکننده‌ی عمومی و مدیر سایت نیز نه در کد فرانت‌اند، بلکه از طریق مکانیزم امنیت سطح ردیف (Row Level Security) و مستقیماً در سطح پایگاه‌داده تعریف و تضمین شده است.

نتیجه‌ی این پروژه، یک پنل مدیریت یکپارچه است که امکان انتشار و ویرایش مقاله، تعدیل نظرات کاربران، مشاهده‌ی آمار و نمودار رفتار بازدیدکنندگان (بازدید صفحه، کلیک دکمه‌ها، مدت حضور)، و مدیریت پیام‌های تماس را در اختیار مدیر سایت قرار می‌دهد؛ در حالی‌که رابط کاربری تمام بخش‌های جدید، از نظر پالت رنگی و الگوهای طراحی، با هویت بصری سایت اصلی کاملاً هماهنگ باقی مانده است.

۱.۱. معرفی پروژه و هدف کلی

وب سایت Marketing-Website یک وب سایت شرکتی است که با هدف معرفی خدمات و جذب مشتری طراحی شده و با استفاده از React، Vite و Tailwind CSS پیاده سازی شده است. در نسخه ی اولیه، این وب سایت صرفاً شامل صفحات معرفی خدمات، نمونه کارها و یک فرم تماس ساده ی بدون ذخیره سازی بود.

هدف این پروژه، ارتقای وب سایت موجود با افزودن سه قابلیت اصلی است که هرکدام یک نیاز مشخص کسب وکار را برطرف می کنند: نخست، یک سیستم بلاگ کامل که امکان انتشار محتوا توسط مدیر سایت و مشاهده و مشارکت (از طریق ثبت نظر) توسط کاربران عمومی را فراهم می کند تا سایت بتواند از طریق محتوای به روز، بازدیدکنندگان بیشتری جذب کند و

رتبه ی بهتری در نتایج موتورهای جستجو کسب نماید. دوم، یک سامانه ی تحلیل رفتار کاربر که بدون وابستگی به ابزارهای شخص ثالث، رفتار واقعی بازدیدکنندگان — شامل صفحات بازدیدشده، دکمه های کلیک شده و مدت زمان حضور در هر صفحه — را ثبت و در قالب نمودار و آمار قابل فهم در اختیار مدیر قرار می دهد تا تصمیمات مبتنی بر داده برای بهبود سایت اتخاذ شود. سوم، یک فرم تماس واقعی و کارآمد که برخلاف نسخه ی قبلی، پیام های ارسالی کاربران را در پایگاه داده ذخیره می کند و به مدیر امکان مشاهده و پاسخ دهی به این پیام ها را از طریق یک پنل اختصاصی می دهد.

هر سه ی این ماژول ها با تکیه بر Supabase به عنوان زیرساخت پایگاه داده و احراز هویت پیاده سازی شده اند، بدون آن که نیاز به توسعه ی یک سرور یا بک اند اختصاصی باشد. این رویکرد باعث شده توسعه ی پروژه سریع تر انجام شود و در عین حال، از طریق مکانیزم امنیت سطح ردیف (Row Level Security)، سطح دسترسی هر بخش از داده به طور دقیق میان بازدیدکننده ی عمومی و مدیر سایت کنترل شود.

۱.۲. مسئله ای که حل می کند

پیش از اجرای این پروژه، وب سایت با سه کاستی اساسی روبه رو بود که هرکدام به طور مستقیم بر توانایی کسب وکار در جذب، شناخت و ارتباط با مشتریان تأثیر منفی می گذاشت.

نبود بلاگ. وب سایت هیچ کانالی برای انتشار محتوای مرتبط با حوزه ی فعالیت کسب وکار نداشت. این کاستی از دو جهت مسئله ساز بود: از منظر سئو، موتورهای جستجو محتوای تازه و به روز را به عنوان معیاری برای رتبه بندی سایت در نظر می گیرند، و نبود چنین محتوایی باعث می شد سایت در نتایج جستجو جایگاه ضعیف تری داشته باشد. از منظر بازاریابی نیز، بلاگ ابزاری برای اثبات تخصص و ایجاد اعتماد نزد مخاطب است؛ بدون آن، کسب وکار فرصت معرفی دانش و تجربه ی خود را به مشتریان بالقوه از دست می داد.

نبود تحلیل رفتار کاربر. مدیر سایت هیچ داده ای درباره ی نحوه ی تعامل بازدیدکنندگان با سایت در اختیار نداشت. مشخص نبود کدام صفحات بیشترین بازدید را دارند، کاربران روی کدام دکمه ها کلیک می کنند، و چه مدت در هر صفحه می مانند. نبود این داده ها تصمیم گیری درباره ی بهبود ساختار سایت، اولویت بندی محتوا، یا سنجش اثربخشی دکمه های فراخوان به اقدام (Call to Action) را عملاً غیرممکن می کرد و هر تغییری در سایت مبتنی بر حدس و گمان انجام می شد، نه شواهد واقعی.

نبود صندوق پیام متمرکز. فرم تماس موجود در سایت، پیام کاربر را در هیچ کجا ذخیره نمی کرد و صرفاً یک پیام «ارسال با موفقیت» به کاربر نمایش می داد. این یعنی در عمل، پیام کاربر پس از ارسال از بین می رفت و هیچ مکانیزمی برای بازیابی، پیگیری یا پاسخ دهی به آن وجود نداشت. این نقص می توانست به از دست رفتن مشتریان بالقوه ای منجر شود که برای درخواست اطلاعات یا همکاری با کسب وکار تماس گرفته بودند اما پاسخی دریافت نمی کردند.

این پروژه با افزودن سه ماژول متناظر — بلاگ، سامانه ی تحلیل رفتار، و فرم تماس متصل به صندوق پیام — هر سه ی این کاستی ها را به طور مستقیم برطرف می کند.

۱.۳. محدوده ی پروژه (Scope)

این پروژه به طور مشخص در چارچوب افزودن سه ماژول به یک وب سایت React موجود تعریف شده و طراحی آن با تمرکز بر نیازهای واقعی و فوری کسب وکار، نه ساخت یک پلتفرم عمومی و همه منظوره، صورت گرفته است. در ادامه مرزهای دقیق محدوده ی پروژه بیان می شود.

آنچه در محدوده ی پروژه قرار دارد:
مدیریت کامل چرخه ی حیات مقاله (ایجاد، ویرایش، حذف، انتشار یا نگه داشتن به صورت پیش نویس) توسط مدیر سایت؛ نمایش عمومی مقالات منتشرشده و امکان ثبت نظر توسط بازدیدکنندگان، همراه با فرآیند تایید نظرات توسط مدیر پیش از نمایش عمومی؛ پشتیبانی از عناصر پایه ای سئو در سطح هر مقاله (عنوان و توضیح متا، متن جایگزین تصویر)؛ ردیابی خودکار و بی واسطه ی بازدید صفحات، مدت زمان حضور در هر صفحه، و کلیک روی دکمه ها و لینک های کلیدی سایت؛ نمایش این داده ها در قالب آمار کلی و نمودار در یک داشبورد اختصاصی؛ ذخیره سازی پیام های ارسالی از فرم تماس در پایگاه داده و امکان مشاهده و پاسخ دهی به آن ها از طریق پنل مدیریت؛ و پیاده سازی یک سامانه ی ورود امن برای مدیر سایت که دسترسی به تمام بخش های مدیریتی را محدود می کند.

آنچه خارج از محدوده ی پروژه قرار دارد:
ارسال خودکار ایمیل برای اطلاع رسانی پاسخ مدیر به کاربر یا اطلاع رسانی نظر جدید؛ هرگونه مکانیزم محدودسازی نرخ درخواست (Rate Limiting) یا کپچا برای جلوگیری از اسپم در فرم های کامنت و تماس؛ ویرایشگر متن گرافیکی (WYSIWYG) برای نگارش مقاله — به جای آن از یک نشانه گذاری متنی ساده برای تیتر و لینک استفاده شده است؛ امکان مدیریت چند سطح دسترسی یا چند مدیر با نقش های متفاوت؛ قابلیت های بلادرنگ (Realtime) مانند به روزرسانی زنده ی داشبورد بدون رفرش صفحه؛ و طراحی یک اپلیکیشن موبایل جداگانه، چرا که تمام رابط های کاربری به صورت واکنش گرا (Responsive) در همان وب سایت پیاده سازی شده اند.

این مرزبندی مشخص، هم به تیم توسعه کمک می کند انتظارات را به درستی مدیریت کند و هم مسیر روشنی برای مراحل بعدی توسعه ی پروژه (که در فصل پایانی این سند فهرست شده اند) فراهم می سازد.

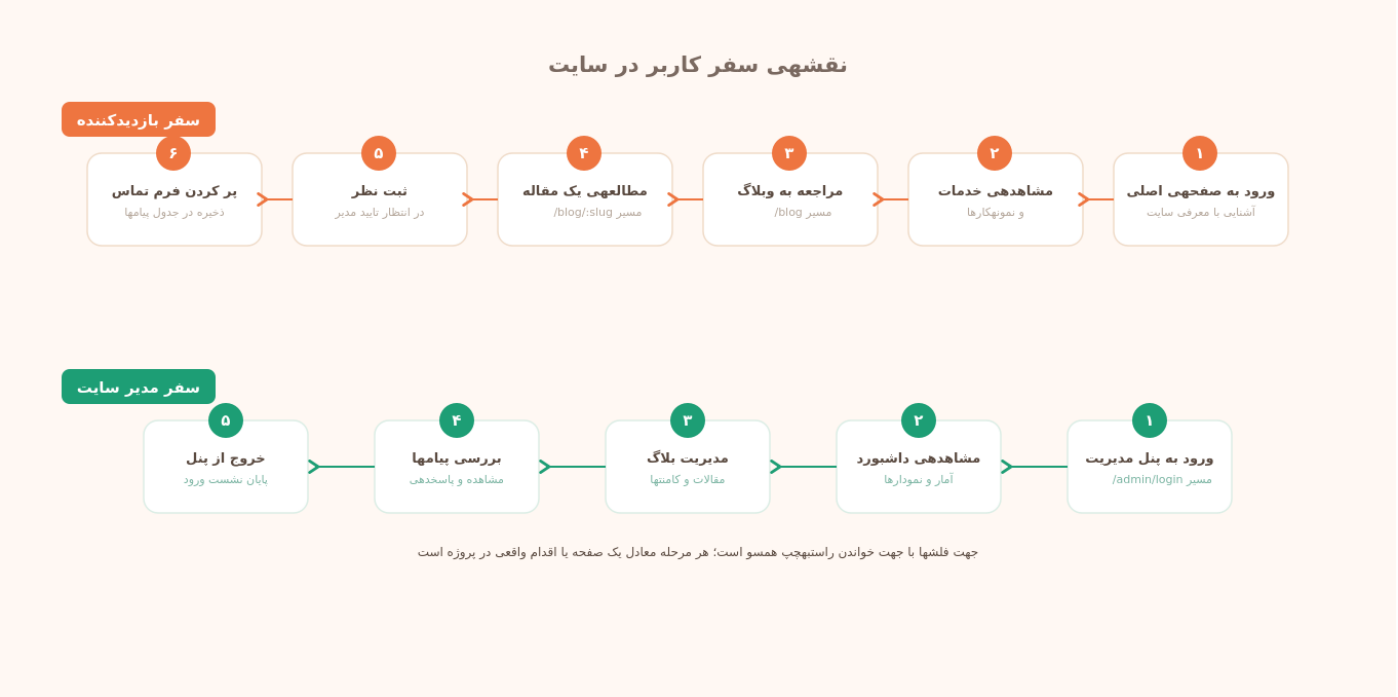
۱.۴. مخاطبان سیستم

سیستم طراحی شده در این پروژه برای دو گروه کاملاً متمایز از کاربران در نظر گرفته شده که هرکدام سطح دسترسی، نیازها، و مسیر تعامل متفاوتی با سایت دارند.

بازدیدکننده ی عمومی. این گروه شامل تمام کاربرانی می شود که بدون نیاز به ثبت نام یا ورود، از بخش های عمومی سایت بازدید می کنند. بازدیدکننده ی عمومی می تواند صفحات معرفی خدمات و نمونه کارهای سایت را مشاهده کند، فهرست مقالات منتشرشده ی بلاگ را ببیند و هر مقاله را به طور کامل مطالعه کند، برای یک مقاله نظر ثبت کند (که پس از تایید مدیر به صورت عمومی نمایش داده می شود)، و از طریق فرم تماس، پیامی برای مدیر سایت ارسال کند. رفتار این گروه از کاربران — بدون آن که خودشان مستقیماً درگیر یا آگاه از این فرآیند باشند — به طور خودکار در قالب داده های بازدید صفحه، مدت حضور، و کلیک روی دکمه ها ثبت می شود تا مبنای تحلیل رفتار کاربر قرار گیرد. از منظر فنی، این گروه در

پایگاه داده با نقش anon (بدون احراز هویت) شناخته می شوند و سطح دسترسی آن ها محدود به عملیات هایی است که RLS به طور صریح برایشان مجاز کرده است.

مدیر سایت. این کاربر که با یک حساب کاربری از پیش تعریف شده وارد سیستم می شود، مسئول مدیریت کامل محتوا و داده های سایت است. مدیر پس از ورود به پنل مدیریت، دسترسی کامل به سه بخش دارد: مدیریت بلاگ (افزودن، ویرایش و حذف مقاله، و همچنین تایید، رد یا حذف نظرات کاربران)، مشاهده ی داشبورد تحلیل رفتار کاربر (شامل آمار کلی، روند بازدید، پرطرفدارترین صفحات و پرکلیک ترین دکمه ها)، و مدیریت صندوق پیام های تماس (مشاهده، علامت گذاری به عنوان خوانده شده، و پاسخ دهی). از منظر فنی، مدیر با نقش authenticated شناخته می شود و به دلیل قوانین RLS تعریف شده، به تمام داده های موجود در تمام جداول دسترسی کامل خواندن و نوشتن دارد. نکته ی قابل توجه این است که چون تمام مسیرهای پنل مدیریت از طریق مکانیزم ProtectedRoute محافظت می شوند، در صورت نبود نشست ورود معتبر، کاربر به طور خودکار به صفحه ی ورود هدایت می شود و امکان دسترسی مستقیم به هیچ یک از بخش های مدیریتی وجود ندارد.



۱.۵. مروری بر راهکارهای آماده و دلیل عدم استفاده از آن‌ها

پیش از تصمیم به پیاده‌سازی اختصاصی هر سه ماژول این پروژه، راهکارهای آماده و شناخته‌شده‌ی موجود در بازار برای هر بخش بررسی و با نیازهای واقعی این پروژه سنجیده شدند.

در حوزه‌ی بلاگ، رایج‌ترین راهکار آماده، WordPress است که به‌عنوان یک سیستم مدیریت محتوای مستقل و کامل، امکانات گسترده‌ای برای انتشار مقاله، مدیریت نظرات، و افزونه‌های سئو ارائه می‌دهد. با این حال، استفاده از WordPress در این پروژه به معنای اجرای یک سیستم کاملاً جدا (با پشته‌ی فنی PHP/MySQL) در کنار وب‌سایت React موجود بود؛ این جداافتادگی یا نیازمند دو زیرساخت مستقل و دو فرآیند دیپلوی جداگانه می‌شد، یا نیازمند تلاش قابل‌توجه برای یکپارچه‌سازی WordPress به‌عنوان یک Headless CMS در پشت فرانت‌اند React، که خود پیچیدگی معماری تازه‌ای

(لایه‌ی میانی برای واکشی داده از WordPress API) اضافه می‌کرد. با توجه به این‌که هدف پروژه افزودن یک بلاگ سبک و کاملاً هماهنگ با پوسته‌ی بصری سایت موجود بود، نه راه‌اندازی یک CMS عمومی و همه‌منظوره، ساخت یک سیستم بلاگ اختصاصی و سبک روی همان پایگاه‌داده‌ی Supabase انتخاب معقول‌تری تشخیص داده شد.

در حوزه‌ی تحلیل رفتار کاربر، رایج‌ترین گزینه، Google Analytics (نسخه‌ی GA٤) است که یک سرویس کامل و رایگان برای ردیابی بازدید، رویداد، و رفتار کاربر ارائه می‌دهد. عدم استفاده از آن در این پروژه بر پایه‌ی سه ملاحظه بود: نخست، داده‌های Google Analytics روی زیرساخت گوگل ذخیره می‌شوند و مستقیماً و بدون واسطه از طریق پایگاه‌داده‌ی پروژه در دسترس نیستند؛ این موضوع در پروژه‌هایی که ملاحظات حریم خصوصی داده یا نیاز به تلفیق مستقیم داده‌ی رفتاری با داده‌های داخلی دیگر (مانند مقالات یا کاربران) دارند، محدودیت ایجاد می‌کند. دوم، رابط گزارش‌گیری Google Analytics عمومی و غیرقابل‌سفارشی‌سازی در پنل مدیریت اختصاصی سایت است؛ در حالی‌که هدف این پروژه، نمایش آمار مستقیماً در همان پنل مدیریتی که مدیر برای بلاگ و پیام‌ها استفاده می‌کند بود. سوم، از نظر آموزشی و فنی، پیاده‌سازی اختصاصی این ماژول امکان درک و کنترل کامل بر نحوه‌ی تعریف رویداد، ساختار داده، و محاسبات آماری را فراهم کرد، که با ماهیت این پروژه (که علاوه بر تحویل محصول، هدف یادگیری معماری BaaS را نیز دنبال می‌کرد) سازگارتر بود.

در حوزه‌ی فرم تماس، سرویس‌های آماده‌ای مانند Formspree یا Getform امکان دریافت پیام فرم‌های HTML را بدون نیاز به بک‌اند فراهم می‌کنند. اما این سرویس‌ها عمدتاً محدود به ارسال ایمیل اعلان هستند و صندوق پیام مدیریتی با قابلیت مشاهده، تغییر وضعیت، و پاسخ‌دهی درون‌برنامه‌ای (که نیاز صریح این پروژه بود) را ارائه نمی‌دهند؛ از سوی دیگر، چون همین پروژه از پیش به Supabase متصل بود، افزودن یک جدول ساده برای پیام‌ها هزینه‌ی توسعه‌ی به‌مراتب کمتری نسبت به یکپارچه‌سازی یک سرویس شخص‌ثالث جداگانه داشت.

در مجموع، انتخاب مشترک در هر سه مورد، اولویت‌دادن به یکپارچگی معماری و کنترل کامل بر داده، به‌جای بهره‌گیری از قابلیت‌های گسترده‌تر اما جداافتاده‌ی ابزارهای عمومی آماده بود؛ تصمیمی که با مقیاس کوچک تا متوسط این پروژه (یک وب‌سایت شرکتی، نه یک پلتفرم بزرگ‌مقیاس) همخوانی داشت.

فصل دوم: معماری کلی و پشته‌ی فنی (Tech Stack)

۲.۱. معماری کلی: فرانت اند مستقیم متصل به Backend-as-a-Service

در معماری سنتی وب اپلیکیشن ها، معمولاً سه لایه ی مجزا وجود دارد: فرانت اند (رابط کاربری)، بک اند (منطق کسب وکار و APIهای سفارشی)، و پایگاه داده. در این مدل، هر درخواست کاربر ابتدا به سرور بک اند می رسد، بک اند منطق لازم (اعتبارسنجی، احراز هویت، قوانین دسترسی) را اجرا می کند، سپس با پایگاه داده ارتباط برقرار می کند و نتیجه را به فرانت اند بازمی گرداند. این مدل نیازمند نوشتن، تست، دیپلوی و نگه داری مستمر یک سرور اختصاصی است.

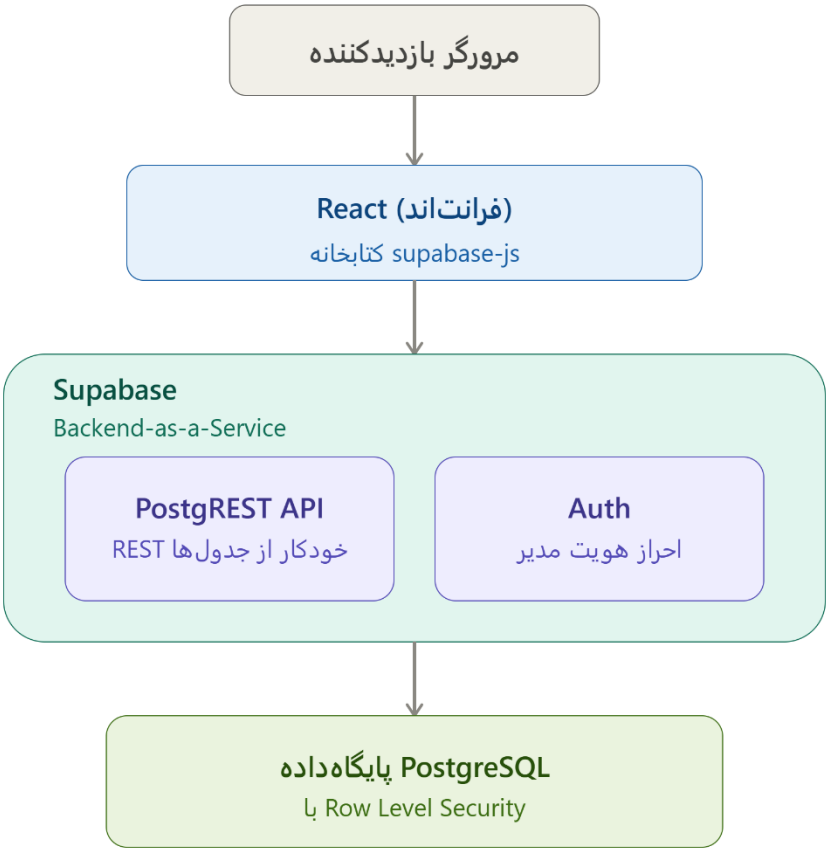
در این پروژه، به جای این معماری سه لایه، از رویکرد Backend-as-a-Service (BaaS) استفاده شده است. در این رویکرد، فرانت اند React مستقیماً و بدون واسطه ی هیچ سروری که خودمان نوشته باشیم، از طریق کتابخانه ی رسمی

@supabase/supabase-js با Supabase ارتباط برقرار می کند. Supabase به ازای هر جدولی که در پایگاه داده ی PostgreSQL تعریف می شود، به طور خودکار یک REST API (از طریق ابزار PostgREST) می سازد؛ به این ترتیب، عملیاتی مانند خواندن مقالات، ثبت کامنت، یا ذخیره ی داده ی ردیابی رفتار کاربر، همگی به صورت درخواست های HTTP مستقیم از مرورگر کاربر به Supabase انجام می شوند.

منطقی که در معماری سنتی باید در کد بک اند نوشته می شد — یعنی «چه کسی اجازه ی دیدن یا تغییر چه داده ای را دارد» — در این معماری به جای کد، به صورت قوانین اعلانی (Declarative) در سطح خود پایگاه داده و از طریق مکانیزم Row Level Security تعریف می شود. برای مثال، به جای نوشتن یک تابع بک اند که بررسی کند «آیا کاربر لاگین کرده تا اجازه ی حذف مقاله داشته باشد یا نه»، این قانون مستقیماً در پایگاه داده به صورت یک Policy تعریف می شود و PostgreSQL خودش در سطح هر کوئری این بررسی را انجام می دهد.

این معماری چند مزیت مستقیم برای این پروژه به همراه داشت: نخست، حذف کامل نیاز به نوشتن و دیپلوی یک سرور Node.js یا مشابه، که زمان توسعه را به طور محسوسی کاهش داد. دوم، کاهش سطح حمله (Attack Surface)، چون هیچ سرور سفارشی ای وجود ندارد که نیاز به وصله ی امنیتی یا نگه داری مستقل داشته باشد. سوم، امکان توسعه ی موازی و سریع فیچرها، چون افزودن یک قابلیت جدید صرفاً نیازمند ساخت یک جدول تازه و تعریف Policy های آن است، نه نوشتن یک لایه ی API کامل. در فصل چهارم این سند، دلایل انتخاب مشخص Supabase (در مقابل سایر گزینه های BaaS یا ساخت بک اند اختصاصی) به طور مفصل بررسی خواهد شد.

۲.۲. نمودار کلی ارتباط اجزا (فرانت اند ↔ Supabase ↔ پایگاه داده)



این نمودار مسیر یک درخواست را از لحظه ی باز شدن سایت تا رسیدن به داده ی واقعی نشان می دهد.

وقتی بازدیدکننده صفحه ای از سایت را باز می کند، مرورگرش کد React را اجرا می کند. این کد، به جای ارسال درخواست به یک سرور بک اند اختصاصی، مستقیماً از طریق کتابخانه ی supabase-js با Supabase صحبت می کند.

Supabaseدر این میان دو نقش موازی ایفا می کند: بخش PostgREST API به ازای هر جدولی که در پایگاه داده تعریف شده (مثلpostscomments، page_views)، یک نقطه ی دسترسی REST خودکار می سازد؛ و بخش Auth مسئول بررسی هویت مدیر هنگام ورود به پنل مدیریت است. هر دوی این ها در نهایت به یک منبع واحد وصل می شوند :پایگاه داده ی.PostgreSQL

نکته ی کلیدی اینجاست که پیش از رسیدن هر درخواست به داده ی واقعی، لایه ی Row Level Security آن را فیلتر می کند. یعنی اینکه یک بازدیدکننده ی عادی فقط مقالات منتشرشده را ببیند، یا فقط مدیرِ لاگین کرده بتواند آمار بازدید را بخواند، نه در کد React بلکه دقیقاً همین جا، در سطح خود پایگاه داده، تضمین می شود. به همین دلیل است که در فصل بعد گفته می شود امنیت این معماری «در دیتابیس» است، نه صرفاً در فرانت اند.

فصل سوم: فریم ورک ها و کتابخانه های فرانت اند

۳.۱. React

۳.۱.۱. معرفی و نسخه ی استفاده شده

Reactیک کتابخانه ی جاوااسکریپت متن باز است که توسط Meta توسعه داده شده و برای ساخت رابط های کاربری تعاملی، به ویژه اپلیکیشن های تک صفحه ای(Single Page Application) ، به کار می رود. ایده ی محوری React این است که رابط کاربری از مجموعه ای از کامپوننت های مستقل و قابل استفاده ی مجدد ساخته می شود؛ هر کامپوننت وضعیت (State) و منطق نمایش خودش را دارد و می تواند مستقل از بقیه ی سیستم توسعه و تست شود. پروژه ی حاضر از نسخه ی ۱۸ React استفاده می کند که شامل قابلیت هایی مانند Concurrent Rendering و مجموعه ی کاملی از Hookها(useState، useEffect ، useMemo و غیره) است — همان ابزارهایی که مستقیماً در ساخت ماژول های بلاگ، تحلیل رفتار، و فرم تماس این پروژه استفاده شده اند.

۳.۱.۲. دلیل انتخاب: معماری کامپوننتی

انتخاب React برای این پروژه دو دلیل اصلی داشت. نخست، یکپارچگی با پایه ی موجود پروژه: وب سایت اصلی از پیش با React ساخته شده بود، و توسعه ی فیچرهای جدید با همان فناوری، از دوباره کاری و ناسازگاری فنی جلوگیری کرد. دوم، و مهم تر، معماری کامپوننتی React اجازه داد ماژول های مختلف پروژه به شکل کاملاً مجزا و قابل نگه داری پیاده سازی شوند. برای نمونه، منطق مدیریت مقالات (BlogAdmin, BlogEditor)، منطق نمایش عمومی بلاگ (BlogList, BlogPost)، و منطق داشبورد آماری (AnalyticsDashboard) هرکدام کامپونت های کاملاً مستقلی هستند که تنها از طریق واسط های مشترک (مانند supabaseClient یا AuthContext) با یکدیگر در ارتباط اند.

این جداسازی چند مزیت عملی در طول توسعه ی پروژه داشت: امکان توسعه و دیباگ هر فیچر به طور مستقل از بقیه (مثلاً رفع خطای فرم مقاله بدون نگرانی از تأثیر روی داشبورد آنالیتیکس)؛ قابلیت استفاده ی مجدد از منطق مشترک از طریق Custom Hookها، مانند usePageTracking که در سطح کل اپلیکیشن یک بار تعریف و در همه جا فراخوانی می شود؛ و مدیریت متمرکز وضعیت های سراسری (مانند وضعیت لاگین مدیر) از طریق React Context، بدون نیاز به کتابخانه های مدیریت وضعیت پیچیده تر مانند Redux که برای مقیاس این پروژه اضافه (Over-engineering) محسوب می شد.

Vite .۳.۲

۳.۲.۱. معرفی به عنوان ابزار Build

Vite(تلفظ: ویت) یک ابزار ساخت (Build Tool) و سرور توسعه (Dev Server) برای پروژه های فرانت اند مدرن است که توسطEvan You ، خالقVue.js ، ساخته شده است. برخلاف باندلرهای سنتی مانند Webpack که کل پروژه را پیش از اجرا باندل می کنند، Viteدر محیط توسعه از قابلیت ماژول های بومی ES (Native ES Modules) مرورگر استفاده می کند و فایل ها را فقط در لحظه ی درخواست، به صورت جداگانه کامپایل و ارسال می کند. برای بیلد نهایی (Production Build)نیز از Rollup در پشت صحنه بهره می برد تا خروجی بهینه و فشرده تولید شود. در این پروژه، Viteهم مسئول

اجرای سرور توسعه (npm run dev) و هم مسئول تولید نسخه ی نهایی برای دیپلوی روی GitHub Pages (npm run build)است.

۳.۲.۲. دلیل انتخاب: سرعت توسعه و Hot Module Replacement

دلیل اصلی انتخاب Vite، تفاوت محسوس آن در سرعت راه اندازی اولیه و سرعت بازتاب تغییرات نسبت به ابزارهای قدیمی تر مانند Create React App بود. از آن جا که این پروژه با روندی کاملاً تکرارشونده و مرحله به مرحله توسعه یافت — هر فایل جداگانه نوشته، ذخیره و بلافاصله در مرورگر بررسی می شد — سرعت بازخورد ابزار توسعه اهمیت مستقیمی بر سرعت کل پروژه داشت.

قابلیت Hot Module Replacement (HMR) در Vite به طور خاص در این فرایند نقش کلیدی داشت: با هر ذخیره ی فایل (چه یک کامپوننت React، چه یک فایل کمکی مانند analytics.js)، فقط همان ماژول مشخص در مرورگر به روزرسانی می شود، بدون آنکه کل صفحه رفرش شود یا وضعیت فعلی برنامه (مانند فرم پرشده یا مسیر باز فعلی) از دست برود. این ویژگی به ویژه هنگام رفع اشکالات نحوی (Syntax Error) در طول توسعه اهمیت پیدا کرد؛ Vite این خطاها را بلافاصله و با جزئیات دقیق (نام فایل، شماره ی خط، و حتی موقعیت دقیق کاراکتر خطا) در قالب یک صفحه ی Overlay در مرورگر نمایش می دهد، که فرایند شناسایی و رفع اشکالاتی مانند تگ های JSX ناقص یا importهای اشتباه را به طور چشمگیری سریع تر کرد.

۳.۳. Tailwind CSS

۳.۳.۱. معرفی رویکرد Utility-First

Tailwind CSSیک فریمورک CSS با رویکرد Utility-First است؛ به این معنا که به جای نوشتن فایل های CSS جداگانه با کلاس های معنایی سفارشی (مانند card-header. یا(btn-primary. ، مجموعه ای بزرگ از کلاس های از پیش تعریف شده و تک منظوره مستقیماً در className عناصر JSX به کار می رود — برای نمونه rounded-۲xl برای گردی گوشه، bg-orange-۶۰۰ برای رنگ پس زمینه، یا flex items-center برای چیدمان. این رویکرد باعث می شود استایل دهی هر عنصر مستقیماً کنار ساختار آن قابل مشاهده باشد و نیاز به جابه جایی مداوم بین فایل JSX و فایل CSS از بین برود. در این پروژه، Tailwindبدون هیچ فایل CSS سفارشی اضافه ای، برای استایل دهی تمام صفحات جدید (فرم لاگین، داشبورد آنالیتیکس، پنل مدیریت بلاگ، صفحات عمومی بلاگ) به کار رفته است.

۳.۳.۲. دلیل انتخاب: هماهنگی با استایل موجود سایت

انتخاب Tailwind برای این پروژه، بیش از هر چیز، تصمیمی برای حفظ یکپارچگی بصری بود، نه صرفاً یک ترجیح فنی مستقل. وب سایت اصلی از پیش به طور کامل با Tailwind ساخته شده بود و شامل یک پالت رنگی مشخص (رنگ های کرم، هلویی و نارنجی، با کد رنگ متن قهوه ای گرم #A۶۹۶۰۷) و الگوهای طراحی ثابت (گوشه های به شدت گرد، سایه های ملایم، دکمه های نارنجی با افکت درخشش هنگام هاور) بود. استفاده از همان فریمورک برای فیچرهای جدید این امکان

را فراهم کرد که این پالت و الگوها را بدون نیاز به تعریف مجدد در یک سیستم استایل دهی متفاوت، مستقیماً بازتولید شود؛ برای نمونه، همان کلاس rounded-۳xl و همان کد رنگ #EEV٥٤٠ که در دکمه ی «Get Started» سایت اصلی استفاده شده بود، عیناً در فرم ورود مدیر و کارت های داشبورد آنالیتیکس نیز به کار رفت.

مزیت عملی دیگر این انتخاب، سرعت پیاده سازی تغییرات ظاهری گسترده بود. در جریان توسعه ی این پروژه، در یک مرحله تصمیم گرفته شد تم تمام صفحات مدیریتی (که ابتدا با پس زمینه ی تیره طراحی شده بودند) به طور کامل به یک تم روشن و پاستیلی هماهنگ با سایت اصلی تغییر کند. از آن جا که تمام استایل ها مستقیماً در کلاس های Tailwind هر کامپوننت تعریف شده بودند (و نه در یک فایل CSS مرکزی)، این تغییر با جایگزینی مستقیم بلوک های JSX هر فایل، بدون نیاز به هماهنگی با یک سیستم theming جداگانه، به سرعت قابل اجرا بود.

۳.۴. React Router DOM

۳.۴.۱. مسیریابی صفحات عمومی و مدیریتی

React Router DOMکتابخانه ی استاندارد و رسمی مسیریابی (Routing) در اکوسیستم React است که امکان تعریف نگاشت میان آدرس های URL و کامپوننت های نمایش داده شده را در یک اپلیکیشن تک صفحه ای (SPA) فراهم می کند؛ یعنی بدون آنکه مرورگر یک درخواست کامل تازه به سرور بفرستد، با تغییر آدرسURL ، Reactکامپوننت متناظر را به جای کامپوننت قبلی رندر می کند. در این پروژه، تمام مسیرهای جدید (هم بخش عمومی بلاگ و هم بخش مدیریتی) در یک فایل مرکزی به نام Routings.jsx تعریف شده اند و از دو کامپوننت اصلی این کتابخانه استفاده می کنند :
<Routes>برای تعریف مجموعه ی مسیرها، و <Route> برای نگاشت هر آدرس به کامپوننت مربوطه. مسیرهای عمومی مانند blog/ و blog/:slug/ بدون هیچ محدودیتی در دسترس هستند، در حالی که مسیرهای مدیریتی مانند admin/analytics/درون یک لایه ی محافظتی قرار گرفته اند که در بخش بعد توضیح داده می شود.

۳.۴.۲. مسیرهای تودرتو (Nested Routes) برای پنل ادمین

یکی از قابلیت های کلیدی React Router که در ساخت پنل مدیریت این پروژه نقش محوری داشت، مسیرهای تودرتو است. به جای تعریف هر صفحه ی مدیریتی (داشبورد آنالیتیکس، مدیریت بلاگ، صندوق پیام ها) به صورت کاملاً مستقل و تکرار کردن سایدبار در هر یک، یک مسیر والد با آدرس admin/ تعریف شده که کامپوننت AdminLayout (شامل سایدبار ثابت و دکمه ی خروج) را رندر می کند؛ و درون همین مسیر، زیرمسیرهایی مانند analytics، blog، و messages به عنوان فرزند تعریف شده اند. محل دقیق نمایش این فرزندان درون AdminLayout توسط کامپوننت <Outlet /> مشخص می شود — هرجا که <Outlet /> در JSX قرار گیرد، محتوای مسیر فرزند فعال دقیقاً همان جا رندر می شود.

این ساختار باعث شد سایدبار و چیدمان کلی پنل مدیریت فقط یک بار نوشته شود و در تغییر بین بخش های مختلف (مثلاً از آنالیتیکس به مدیریت بلاگ)، فقط ناحیه ی محتوای اصلی تغییر کند، بدون رندر مجدد کل صفحه. علاوه بر این، محافظت از دسترسی نیز در همین سطح متمرکز شد: کافی است کامپوننت AdminLayout درون یک ProtectedRoute قرار

گیرد تا تمام مسیرهای فرزند آن به طور خودکار محافظت شده باشند، بدون نیاز به تکرار منطق بررسی ورود در هر صفحه ی مدیریتی به طور جداگانه.

۳.۵. React Hook Form

۳.۵.۱. مدیریت فرم تماس

React Hook Formکتابخانه ای برای مدیریت فرم ها در React است که برخلاف رویکرد رایج مبتنی بر) useState که برای هر فیلد فرم یک متغیر وضعیت جداگانه و یک تابع onChange تعریف می شود)، از یک سیستم مبتنی بر رجیستری فیلد (Uncontrolled Components) استفاده می کند. با فراخوانی هوک()useForm ، دو ابزار اصلی در اختیار قرار می گیرد: تابع register که هر ورودی فرم را به سیستم مدیریت داخلی کتابخانه متصل می کند، و تابع handleSubmit که پیش از اجرای منطق ارسال، به طور خودکار اعتبارسنجی فیلدها را بررسی می کند.

در این پروژه، React Hook Form مستقیماً برای مدیریت فرم تماس موجود در کامپوننت Contact.jsx به کار رفته است. هر فیلد (نام، ایمیل، موضوع، متن پیام) با {...(register("fieldName", { required: true }} به فرم متصل شده، به این معنا که کتابخانه به طور خودکار بررسی می کند این فیلدها پیش از ارسال خالی نمانده باشند. با فراخوانی handleSubmit(onSubmit)، تابع onSubmit فقط زمانی اجرا می شود که تمام قوانین اعتبارسنجی برآورده شده باشند؛ در غیر این صورت، مرورگر به طور پیش فرض روی فیلد ناقص متوقف می شود، بدون آنکه نیازی به نوشتن منطق دستی بررسی خطا در کد پروژه باشد. داده ی نهایی و معتبرشده ی فرم مستقیماً از پارامتر data تابع onSubmit در اختیار قرار می گیرد و برای ذخیره سازی به جدول messages در Supabase ارسال می شود.

استفاده از این کتابخانه به جای مدیریت دستی وضعیت فرم، حجم کد کامپوننت Contact.jsx را کاهش داد و منطق اعتبارسنجی پایه (مانند اجباری بودن فیلدها) را بدون نوشتن کد تکراری برای هر فیلد فراهم کرد؛ ضمن آنکه چون این کتابخانه پیش از این پروژه نیز در بخش های دیگر سایت اصلی مورد استفاده بود، از افزودن یک وابستگی جدید و ناآشنا به پروژه پرهیز شد.

۳.۶. React Toastify

۳.۶.۱. نمایش پیام های موفقیت/خطا

React Toastifyکتابخانه ای برای نمایش اعلان های موقت و غیرمسدودکننده (Toast Notifications) در گوشه ی صفحه است — همان نوار کوچک رنگی که پس از انجام یک عملیات (مانند ارسال موفق فرم) برای چند ثانیه ظاهر می شود و سپس خودکار محو می شود، بدون آنکه جریان عادی کار کاربر را متوقف کند (برخلاف alert() بومی مرورگر که تعامل با صفحه را کاملاً مسدود می کند). راه اندازی این کتابخانه در سطح کل اپلیکیشن انجام می شود: کامپوننت <ToastContainer /> یک بار در فایل ریشه (main.jsx) و در کنار <App /> قرار می گیرد، و از آن پس، در هر نقطه

از پروژه می توان با فراخوانی توابعی مانند toast.success(...) یا toast.error(...) یک اعلان جدید نمایش داد، بدون نیاز به مدیریت وضعیت نمایش/عدم نمایش آن به صورت دستی.

در این پروژه، React Toastify مشخصاً در فرم تماس (Contact.jsx) به کار رفته است: پس از تلاش برای ذخیره ی پیام در جدول messages، اگر عملیات با خطا مواجه شود، toast.error("ارسال پیام با خطا مواجه شد. دوباره تلاش کنید.") نمایش داده می شود؛ و در صورت موفقیت، toast.success("پیام شما با موفقیت ارسال شد") به کاربر بازخورد مثبت می دهد و همزمان فرم با reset() پاک سازی می شود. این الگو یک تجربه ی کاربری واضح و فوری فراهم می کند: کاربر بدون نیاز به رفرش صفحه یا حدس زدن نتیجه ی عملیات، بلافاصله می فهمد که آیا پیامش ثبت شده یا خیر. از آن جا که React Toastify پیش از این پروژه نیز بخشی از وابستگی های سایت اصلی بود، استفاده از آن برای این فیچر جدید، به جای معرفی یک کتابخانه ی اعلان جایگزین، انتخابی طبیعی برای حفظ یکپارچگی تجربه ی کاربری در سراسر سایت بود.

۳.۷. React Icons

React Icons مجموعه ای گسترده از آیکون های برداری (SVG) از چندین کتابخانه ی معروف) از جمله Font Awesome، Material Design، Ionicons، و (Remix Icon است که در قالب کامپوننت های آماده ی React بسته بندی شده اند. برخلاف روش سنتی استفاده از فونت های آیکون یا فایل های SVG جداگانه، هر آیکون در این کتابخانه صرفاً یک کامپوننت React است که می توان آن را مانند هر عنصر دیگر ایمپورت کرد (برای نمونه import { RxHamburgerMenu } from "react-icons/rx") و به راحتی رنگ، اندازه و سایر ویژگی هایش را از طریق props یا کلاس های Tailwind کنترل کرد.

این کتابخانه پیش از این پروژه در سایت اصلی برای آیکون های نوار ناوبری (مانند آیکون منوی همبرگری) و بخش تماس با ما (آیکون های موقعیت مکانی، تلفن و ایمیل) به کار رفته بود و در این پروژه بدون تغییر همان الگو ادامه یافت. دلیل اصلی ادامه ی استفاده از آن، یکپارچگی بصری و پرهیز از افزودن یک وابستگی جدید برای نیازی است که از پیش پوشش داده شده بود؛ به طور خاص، در پیاده سازی این پروژه نیازی به آیکون های تازه در بخش های جدید (بلاگ، آنالیتیکس، پیام ها) احساس نشد، چون این صفحات عمدتاً از عناصر متنی و کارت های ساده تشکیل شده اند.

۳.۸. Recharts

۳.۸.۱. دلیل انتخاب: نمودارسازی مبتنی بر کامپوننت React

Rechartsیک کتابخانه ی نمودارسازی است که بر پایه ی SVG ساخته شده و به طور خاص برای اکوسیستم React طراحی شده است. تفاوت اصلی آن با کتابخانه های نمودارسازی عمومی تر) مانند Chart.js که بر پایه ی Canvas و دستکاری امری DOM کار می کند (در این است که هر نمودار در Recharts نیز خودش از مجموعه ای از کامپوننت های Reactترکیب می شود — برای نمونه<LineChart>، <XAxis>، <YAxis> ، <Tooltip> و <Line> همگی کامپوننت های جداگانه ای هستند که به صورت تودرتو در JSX قرار می گیرند، دقیقاً به همان سبکی که بقیه ی رابط کاربری این پروژه

نوشته شده است. این هماهنگی معماری، یادگیری و استفاده از کتابخانه را برای تیمی که از پیش با React کار می کند، طبیعی و بدون منحنی یادگیری اضافه می کند.

۳.۸.۲. استفاده در داشبورد تحلیل رفتار کاربر

Rechartsمشخصاً و منحصراً در کامپوننت AnalyticsDashboard.jsx برای نمایش گرافیکی داده های رفتار کاربر به کار رفته است. سه نمودار مجزا با این کتابخانه ساخته شده اند: یک نمودار خطی (LineChart) برای نمایش روند بازدید روزانه ی سایت در طول بازه ی زمانی انتخاب شده؛ و دو نمودار میله ای افقی BarChart) با ("layout="vertical برای نمایش پرطرفدارترین صفحات سایت و پرکلیک ترین دکمه ها. داده ی خام این نمودارها از جداول page_views و eventsدر Supabase خوانده می شود و پیش از رسم، با استفاده از هوک useMemo در سمت کلاینت پردازش و تجمیع (Aggregate) می شود — یعنی محاسباتی مانند شمارش تعداد بازدید هر صفحه یا میانگین گیری از مدت حضور، در خود مرورگر و نه در سطح پایگاه داده انجام می شود، که برای حجم داده ی این پروژه (یک سایت شرکتی با ترافیک متوسط) رویکردی کافی و بدون نیاز به پیچیدگی کوئری های تجمیعی سمت سرور است. کامپوننت ResponsiveContainerنیز اطمینان می دهد این نمودارها در اندازه های مختلف صفحه نمایش (از موبایل تا دسکتاپ) به درستی مقیاس بندی شوند.

۳.۹.۱. معرفیAOS

AOS (Animate On Scroll)یک کتابخانه ی سبک جاوااسکریپت است که به عناصر HTML امکان می دهد هنگام ورود به دید کاربر در حین اسکرول صفحه (مانند محو شدن تدریجی یا حرکت از یک جهت)، به طور خودکار انیمیشن بگیرند؛ فعال سازی آن معمولاً تنها با افزودن یک ویژگی "data-aos="fade-up به تگ موردنظر و فراخوانی یک باره ی ()AOS.initدر سطح کامپوننت انجام می شود، بدون نیاز به نوشتن منطق انیمیشن سفارشی.

GSAP (GreenSock Animation Platform)در مقابل، یک کتابخانه ی انیمیشن قدرتمندتر و سطح پایین تر است که کنترل دقیق تری بر زمان بندی، توالی، و ترکیب چند انیمیشن هم زمان (Timeline) فراهم می کند و معمولاً برای جلوه های پیچیده تر یا تعاملی تر (فراتر از صرفِ نمایان شدن هنگام اسکرول) به کار می رود.

۳.۹.۲. جایگاه در این پروژه

هر دوی این کتابخانه ها پیش از شروع این پروژه، بخشی از وابستگی های وب سایت اصلی بودند و برای جلوه های بصری صفحات معرفی خدمات و نمونه کارها (مانند محو و ظاهرشدن تدریجی بخش های مختلف هنگام اسکرول کاربر در صفحه ی اصلی) استفاده می شدند. در طول این پروژه، هیچ استفاده ی جدیدی از GSAP صورت نگرفت، اما AOS در یک نقطه ی مشخص و به طور محدود به کد جدید نیز راه یافت: کامپوننت Contact.jsx (که فرم تماس آن به دیتابیس متصل شد) از پیش شامل فراخوانی AOS.init() و ویژگی های data-aos روی بخش های نقشه و فرم بود، و این الگو در بازنویسی این کامپوننت برای اتصال به Supabase، بدون تغییر حفظ شد تا جلوه ی بصری موجود آن مختل نشود.

نکته ی قابل توجه این است که هیچ کدام از صفحات کاملاً تازه ساز این پروژه — یعنی صفحات بلاگ، پنل مدیریت، داشبورد آنالیتیکس، و صندوق پیام ها — از AOS یا GSAP استفاده نمی کنند. تصمیم آگاهانه این بود که این صفحات با انیمیشن های ورودی حداقلی طراحی شوند، چون ماهیت کاربردی و مدیریتی آن ها (در مقابل ماهیت بازاریابی و معرفی محور صفحه ی اصلی) نیاز کمتری به جلوه های بصری تزئینی دارد؛ در عوض، تعاملات ظریف تری مانند transition-all duration-۳۰۰ در Tailwind (برای افکت هاور روی دکمه ها و کارت ها) جایگزین آن ها شدند.

فصل چهارم: پایگاه داده و Supabase

۴.۱. معرفی Supabase به عنوان Backend-as-a-Service

Supabaseیک پلتفرم متن باز از دسته ی Backend-as-a-Service (BaaS) است که مجموعه ای از خدمات زیرساختی مورد نیاز یک اپلیکیشن وب یا موبایل — پایگاه داده، احراز هویت، ذخیره سازی فایل، وAPI های خودکار — را بدون نیاز به نوشتن و مدیریت یک سرور بک اند اختصاصی در اختیار توسعه دهنده قرار می دهد. برخلاف بسیاری از پلتفرم های BaaS رقیب) مانند (Firebase که از پایگاه داده های NoSQL استفاده می کنند، هسته ی Supabase یک نمونه ی کامل و واقعی از PostgreSQL است؛ یعنی توسعه دهنده به تمام قابلیت های یک پایگاه داده ی رابطه ای بالغ) جدول های مرتبط، تراکنش، محدودیت های یکتایی، ایندکس گذاری، و زبان کوئری SQL استاندارد (دسترسی کامل دارد، نه یک زیرمجموعه ی ساده شده ی آن.

Supabaseحول چهار سرویس اصلی سازمان دهی شده که در این پروژه، سه ی آن ها به طور مستقیم مورد استفاده قرار گرفته اند) Database :خود پایگاه داده ی PostgreSQL که تمام جدول های پروژه، از posts تاpage_views ، در آن تعریف شده اند(، Auth (سرویس احراز هویتی که برای پیاده سازی ورود مدیر سایت به کار رفت، بدون نیاز به ساخت سیستم رمزنگاری و مدیریت نشست از پایه)، و) Auto-generated REST API لایه ای موسوم به PostgREST که بلافاصله پس از ساخت هر جدول در پایگاه داده، بدون نوشتن حتی یک خط کد اضافی، یک نقطه ی دسترسی REST کامل برای عملیات خواندن، نوشتن، ویرایش و حذف روی آن جدول فراهم می کند .(سرویس چهارم Supabase یعنی Storage(ذخیره سازی فایل)، در این پروژه استفاده نشده، چرا که تصاویر کاور مقالات از طریق لینک مستقیم URL) خارجی (وارد می شوند، نه آپلود مستقیم فایل.

از منظر معماری، این معرفی زمینه ی فصل های بعدی این سند را فراهم می کند: بخش ۴.۲ به دلایل مشخص انتخاب Supabase برای این پروژه (در مقابل جایگزین هایی مانند ساخت یک بک اند سفارشی با Node.js یا استفاده از Firebase) می پردازد، و بخش های بعدی، نحوه ی پیاده سازی عملی جدول ها، روابط، و سیاست های امنیتی را در همین پلتفرم شرح می دهند.

۴.۲. چرا از Supabase استفاده شد؟

۴.۲.۱. عدم نیاز به توسعه و نگه داری یک بک اند اختصاصی

اگر این پروژه با معماری سنتی پیاده سازی می شد، پیش از نوشتن حتی یک خط کد مربوط به فیچرهای اصلی (بلاگ، آنالیتیکس، فرم تماس)، لازم بود یک سرور بک اند مستقل (برای نمونه با Node.js و Express) ساخته می شد که مسئول تعریف Endpointها، اتصال به پایگاه داده، و اجرای منطق کسب وکار باشد. این سرور پس از توسعه نیز نیازمند میزبانی (Hosting)، پایش، به روزرسانی وابستگی ها، و مدیریت مستقل چرخه ی عمر خودش می بود. Supabase این لایه ی کامل را حذف کرد: فرانت اند مستقیماً و بدون واسطه با پایگاه داده صحبت می کند، و آنچه در معماری سنتی «کد بک اند» نامیده می شد، در این پروژه به «تعریف جدول و قانون دسترسی در پایگاه داده» تبدیل شد.

۴.۲.۲. پایگاه داده ی PostgreSQL واقعی زیر پوسته

انتخاب یک BaaS مبتنی بر PostgreSQL (به جای گزینه های NoSQL) به این معناست که پروژه از روز اول به ابزارهای رابطه ای استاندارد و اثبات شده دسترسی داشت: تعریف کلید خارجی با references (مانند ارتباط جدول comments به posts)، محدودیت unique برای جلوگیری از تکرار Slug مقالات، و check برای محدود کردن مقادیر مجاز یک ستون (مانند وضعیت published/draft مقالات). این ابزارها بدون نیاز به کد اضافی در فرانت اند، صحت داده را در سطح خود پایگاه داده تضمین می کنند.

۴.۲.۳. تولید خودکار REST API از روی جدول ها

هر جدولی که در Supabase تعریف می شود، بلافاصله و بدون نوشتن هیچ کد سمت سرور اضافه ای، یک مجموعه ی کامل از عملیات REST (خواندن، فیلترکردن، مرتب سازی، درج، ویرایش، حذف) در اختیار قرار می دهد که از طریق کتابخانه ی supabase-js با یک نحو ساده و زنجیره ای (مانند
supabase.from("posts").select("*").eq("status", "published")) در دسترس است. این ویژگی زمان لازم برای افزودن هر فیچر جدید را به طور چشمگیری کاهش داد؛ برای نمونه، افزودن جدول messages برای فرم تماس، بلافاصله و بدون هیچ مرحله ی میانی، امکان ذخیره و بازیابی پیام ها را از فرانت اند فراهم کرد.

۴.۲.۴. سیستم احراز هویت آماده

پیاده سازی صحیح و امن یک سیستم ورود کاربر از پایه — شامل هش کردن رمز عبور، مدیریت نشست (Session)، و جلوگیری از حملات رایج — کاری زمان بر و مستعد خطاست. سرویس Auth سوپابیس این نیاز را به طور کامل پوشش داد: ساخت حساب مدیر از طریق پنل Supabase انجام شد، و در سمت کد، تنها با فراخوانی
supabase.auth.signInWithPassword() و بررسی نشست فعال، کل فرایند ورود مدیر پیاده سازی شد.

۴.۲.۵. امنیت سطح ردیف برای کنترل دقیق دسترسی

Row Level Securityامکان تعریف قوانین دقیق در سطح هر ردیف از داده را، مستقیماً در پایگاه داده و مستقل از کد فرانت اند، فراهم کرد. این ویژگی به طور خاص برای این پروژه حیاتی بود، چون فرانت اند مستقیماً و بدون لایه ی واسط به

پایگاه داده متصل است؛ بدونRLS ، هر بازدیدکننده ی سایت نظری می توانست از طریق کنسول مرورگر مستقیماً به تمام داده های حساس (مانند پیام های تماس یا آمار خام بازدید) دسترسی داشته باشد.

۴.۲.۶. پلن رایگان مناسب پروژه های کوچک تا متوسط

سطح رایگان Supabase (شامل فضای پایگاه داده، تعداد درخواست، و ظرفیت Auth) برای مقیاس یک وب سایت شرکتی با ترافیک متوسط کاملاً کافی بود و امکان توسعه و آزمایش کامل پروژه را بدون هیچ هزینه ی زیرساختی فراهم کرد.

۴.۲.۷. کاهش زمان توسعه (Time-to-Market)

مجموع موارد بالا باعث شد زمان لازم از «تصمیم به افزودن یک فیچر» تا «فیچر قابل استفاده» به طور چشمگیری کوتاه شود. برای نمونه، کل فرایند افزودن ماژول فرم تماس — از تعریف جدول تا اتصال فرم و ساخت صندوق پیام مدیریتی — تنها در چند مرحله ی متوالی و بدون نیاز به راه اندازی زیرساخت جدید انجام شد.

۴.۲.۸. قابلیت مقیاس پذیری در صورت رشد پروژه

از آن جا که Supabase بر پایه ی PostgreSQL استاندارد بنا شده (نه یک فناوری اختصاصی و بسته)، در صورت رشد آینده ی پروژه، امکان ارتقا به پلن های بالاتر Supabase، یا حتی در نهایت مهاجرت پایگاه داده به یک نمونه ی PostgreSQL خودمیزبان، بدون نیاز به بازنویسی کامل ساختار داده وجود دارد — مزیتی که در پلتفرم های BaaS با فناوری اختصاصی معمولاً وجود ندارد.

۴.۳. به چه دردی می خورد؟ (کاربردهای عملی در این پروژه)

اگر بخش قبل به این پرداخت که چرا Supabase از منظر معماری و مزایای کلی انتخاب شد، این بخش نشان می دهد که این ابزار در عمل و به طور مشخص، چه مسئله ای را در هر یک از سه ماژول این پروژه حل کرده است.

۴.۳.۱. ذخیره سازی مقالات و کامنت ها

پیش از این پروژه، سایت هیچ مکانیزمی برای نگه داری محتوای پویا نداشت — هر تغییری در متن سایت نیازمند ویرایش مستقیم کد و بیلد مجدد بود. Supabase این مشکل را با فراهم کردن دو جدول ساده (posts و comments) حل کرد که اکنون مدیر می تواند بدون نیاز به دانش برنامه نویسی یا دسترسی به کد، از طریق فرم های پنل مدیریت، مقاله ی جدید منتشر کند یا محتوای موجود را ویرایش کند؛ و کاربران عمومی می توانند بدون نیاز به ثبت نام، روی مقالات نظر بگذارند. جریان تایید نظرات (وضعیت pending) نیز مستقیماً از طریق یک ستون ساده در همین جدول، بدون نیاز به هیچ سیستم تعدیل محتوای پیچیده، پیاده سازی شد.

۴.۳.۲. ذخیره سازی داده ی خام رفتار کاربران

ماژول تحلیل رفتار کاربر ذاتاً نیازمند ثبت مداوم و حجیم رویدادهای کوچک است: هر بازدید صفحه، هر کلیک، و هر تغییر مسیر. بدون یک پایگاه داده در دسترس، این داده ها اصلاً قابل جمع آوری نبودند. سه جدول sessions، page_views، و events این نقش را ایفا می کنند: هرکدام از این جداول به محض وقوع رویداد مربوطه (باز شدن یک صفحه ی جدید، خروج از یک صفحه، کلیک روی یک دکمه)، مستقیماً از فرانت اند و بدون هیچ تأخیر یا نیاز به پردازش میانی، در پایگاه داده ثبت می شوند. این داده ی خام سپس در داشبورد آنالیتیکس خوانده، تجمیع، و به آماری قابل فهم (مانند «پرطرفدارترین صفحات») تبدیل می شود.

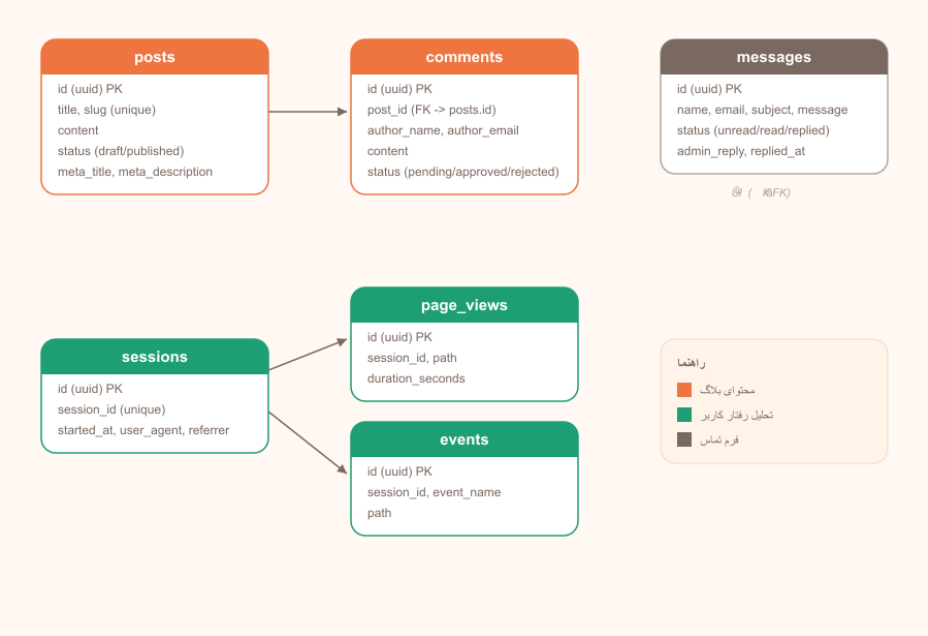
۴.۳.۳. ذخیره سازی پیام های فرم تماس

پیش از این پروژه، پیام های فرم تماس پس از ارسال بلافاصله از بین می رفتند. جدول messages این مشکل را با ذخیره ی دائمی هر پیام (همراه با وضعیتی که نشان می دهد پیام خوانده شده یا پاسخ داده شده یا نه) برطرف کرد. این جدول اکنون پایه ی صندوق پیام مدیریتی است — بدون آن، اصلاً هیچ سابقه ای از پیام های ارسالی کاربران وجود نمی داشت که بتوان به آن ها رجوع کرد.

۴.۳.۴. احراز هویت مدیر سایت

از آن جا که تمام سه ماژول این پروژه شامل بخش های مدیریتی حساس هستند (حذف مقاله، خواندن آمار خصوصی، مشاهده ی پیام های کاربران)، نیاز به یک لایه ی ورود امن برای تشخیص «مدیر» از «بازدیدکننده ی عادی» وجود داشت. سرویس Auth سوپابیس دقیقاً همین نیاز را برطرف کرد: با یک حساب کاربری واحد که در بخش Authentication پنل Supabase تعریف شده، مدیر می تواند وارد پنل شود، و این نشست ورود (Session) است که اساس تمام تصمیمات دسترسی در سراسر پروژه — از نمایش یا عدم نمایش دکمه های ویرایش تا اجرا یا رد شدن قوانین Row Level Security در پایگاه داده — قرار می گیرد.

۴.۴. ساختار جدول ها



۴.۴.۱. جدول posts

جدول posts مرکز ماژول بلاگ است و هر ردیف آن معادل یک مقاله است. ستون id از نوع uuid به عنوان کلید اصلی، شناسه ی داخلی و غیرقابل حدس هر مقاله را تعیین می کند. ستون title عنوان نمایشی مقاله را نگه می دارد، در حالی که ستون slug — با محدودیت unique — نسخه ی خوانا و مناسب URL آن است (برای نمونه my-first-post) که در آدرس عمومی مقاله (/blog/my-first-post) به کار می رود؛ این دو ستون عمداً از هم جدا شده اند تا عنوان بتواند شامل فاصله، حروف بزرگ یا نویسه های فارسی باشد، بدون آنکه بر ساختار URL تأثیر بگذارد. ستون content متن کامل مقاله را با نشانه گذاری ساده (شرح در فصل ششم) نگه می دارد. دو ستون cover_image_url و cover_image_alt به ترتیب لینک تصویر کاور و متن جایگزین آن برای سئو و دسترسی پذیری را ذخیره می کنند. ستون های meta_title و meta_description عناصر سئوی اختصاصی هر مقاله هستند که در صورت خالی بودن، به طور پیش فرض از عنوان اصلی مقاله استفاده می شود. ستون status با محدودیت check تنها دو مقدار draft (پیش نویس) یا published (منتشرشده) را می پذیرد و تعیین می کند مقاله برای عموم قابل مشاهده باشد یا نه. در نهایت، created_at و updated_at به ترتیب زمان ایجاد و آخرین ویرایش مقاله را با نوع timestamptz ثبت می کنند.

۴.۴.۲. جدول comments

جدول comments نظرات ثبت شده روی مقالات را نگه می دارد و از طریق ستون post_id — که با references posts(id) و on delete cascade تعریف شده — به جدول posts مرتبط است؛ این یعنی اگر مقاله ای حذف شود، تمام کامنت های وابسته به آن نیز به طور خودکار و بدون نیاز به کوئری جداگانه حذف می شوند. از آن جا که سیستم کامنت این پروژه نیازی به ثبت نام کاربر ندارد، هویت نویسنده ی هر کامنت صرفاً از طریق دو ستون متنی author_name و author_email ثبت می شود. ستون content متن کامنت را نگه می دارد، و ستون status — با مقدار پیش فرض pending — مکانیزم تعدیل محتوا را پیاده سازی می کند: هر کامنت تازه ارسال شده تا پیش از بررسی و تغییر وضعیت به approved توسط مدیر، به طور عمومی نمایش داده نمی شود (وضعیت rejected نیز برای کامنت های ردشده در نظر گرفته شده است).

۴.۴.۳. جدول messages

جدول messages معادل پایگاه داده ای فرم تماس سایت است. چهار ستون نخست آن — name، email، subject، و message — مستقیماً با فیلدهای فرم تماس در Contact.jsx نگاشت می شوند. ستون status، با سه مقدار مجاز unread، read، و replied، چرخه ی حیات یک پیام را از لحظه ی دریافت تا پاسخ دهی دنبال می کند و پایه ی نشان گر تعداد «پیام خوانده نشده» در پنل مدیریت است. دو ستون admin_reply و replied_at پاسخ متنی مدیر و زمان ثبت آن پاسخ را نگه می دارند؛ لازم به ذکر است این پاسخ صرفاً درون پنل مدیریت ذخیره می شود و ارسال خودکار آن به ایمیل کاربر در محدوده ی این نسخه از پروژه قرار ندارد.

۴.۴.۴. جدول sessions

جدول sessions نقطه ی شروع زنجیره ی ردیابی رفتار کاربر است. به ازای هر بازدیدکننده ی تازه (که در مرورگرش هیچ شناسه ی نشست ذخیره شده ای موجود نباشد)، یک ردیف جدید در این جدول ساخته می شود. ستون session_id یک شناسه ی یکتا و تصادفی (تولیدشده با crypto.randomUUID() در سمت مرورگر) است که در sessionStorage مرورگر کاربر نیز ذخیره می شود تا در طول همان نشست مرورگر پایدار بماند. ستون های started_at و last_seen_at بازه ی زمانی فعالیت بازدیدکننده را مشخص می کنند، و ستون های user_agent و referrer به ترتیب مشخصات مرورگر/دستگاه و صفحه ی ارجاع دهنده ی کاربر (مثلاً یک موتور جستجو یا شبکه ی اجتماعی) را برای تحلیل های تکمیلی احتمالی ذخیره می کنند.

۴.۴.۵. جدول page_views

جدول page_views هسته ی اصلی داده ی نمایش داده شده در داشبورد آنالیتیکس است. هر ردیف این جدول معادل یک بار «دیدن یک صفحه» توسط یک بازدیدکننده است. ستون session_id (بدون تعریف کلید خارجی رسمی، برای جلوگیری از وابستگی سخت گیرانه در صورت تأخیر جزئی در ثبت نشست) این بازدید را به بازدیدکننده ی مربوطه مرتبط می کند. ستون path آدرس دقیق صفحه ی بازدیدشده (مانند /blog یا /) را ذخیره می کند. ستون duration_seconds — که مقدار اولیه اش خالی است — پس از خروج کاربر از آن صفحه، با محاسبه ی فاصله ی زمانی میان ورود و خروج، به روزرسانی می شود؛ همین ستون است که مبنای محاسبه ی «میانگین مدت حضور» در داشبورد قرار می گیرد.

۴.۴.۶. جدول events

جدول events رویدادهای گسسته ای مانند کلیک روی دکمه ها و لینک های مهم را ثبت می کند و از نظر ساختاری مشابه page_views است، با این تفاوت که به جای یک بازه ی زمانی، صرفاً یک لحظه ی وقوع را نشان می دهد. ستون event_name نام قابل خواندن رویداد را نگه می دارد (برای نمونه cta_click:Get Started یا nav_click:About)، که این قالب نام گذاری اجازه می دهد در داشبورد، رویدادها بر اساس نام دقیقشان دسته بندی و شمارش شوند. ستون path نیز مشخص می کند این کلیک در کدام صفحه از سایت رخ داده است، که امکان تحلیل رفتار کاربر را نه فقط در سطح کل سایت، بلکه به تفکیک هر صفحه نیز فراهم می کند.

۴.۵. ارتباطات بین جداول (Foreign Keys)

از میان شش جدول این پروژه، فقط دو رابطه ی کلید خارجی رسمی (Foreign Key) در پایگاه داده تعریف شده و بقیه ی جداول عمداً بدون چنین قیدی طراحی شده اند. این تصمیم نه یک سهل انگاری، بلکه انتخابی آگاهانه بر اساس ماهیت هر رابطه بوده است.

تنها رابطه ای است که با comments.post_id → posts.id به طور رسمی در سطح پایگاه داده تعریف شده، به همراه قید references posts(id) on delete cascade. انتخاب این قید به این معناست که حذف یک مقاله، به طور خودکار تمام کامنت های وابسته به آن را نیز حذف می کند — منطقی که کاملاً با معنای واقعی داده همخوان است: کامنتی که

مقاله اش دیگر وجود ندارد، بی معناست و نگه داشتنش صرفاً داده ی یتیم (Orphaned Data) در پایگاه داده باقی می گذارد.

در مقابل، رابطه ی میان sessions.session_id و دو جدول page_views و events عمداً به صورت کلید خارجی رسمی تعریف نشده است، هرچند از نظر منطقی هر بازدید صفحه یا رویداد متعلق به یک نشست مشخص است. دلیل این تصمیم، ماهیت غیرهمزمان (Asynchronous) ثبت این داده هاست: وقتی بازدیدکننده ای وارد سایت می شود، دو درخواست شبکه تقریباً هم زمان ارسال می شود — یکی برای ساخت ردیف sessions و دیگری برای ثبت اولین page_views. اگر یک قید کلید خارجی سخت گیرانه وجود داشت، در شرایطی که (به هر دلیل شبکه ای) درخواست ثبت page_views اندکی زودتر از تکمیل ثبت sessions به پایگاه داده برسد، کل عملیات با خطا رد می شد و آن بازدید هرگز ثبت نمی شد. حذف این قید رسمی، به قیمت از دست دادن تضمین سخت گیرانه ی یکپارچگی ارجاعی (Referential Integrity)، قابلیت اطمینان عملی ثبت داده ی ردیابی را افزایش داد — انتخابی که با توجه به ماهیت این داده (آمار تحلیلی، نه رکورد تراکنشی حساس) توجیه پذیر است.

جدول messages نیز به هیچ جدول دیگری متصل نیست، چرا که هر پیام تماس، رکوردی کاملاً مستقل و بدون وابستگی منطقی به مقاله، کامنت، یا نشست خاصی است.

۴.۶. سیاست های امنیتی (RLS Policies)

۴.۶.۱. تفاوت نقش anon و authenticated

Supabaseهر درخواستی که به پایگاه داده می رسد را بر اساس کلید احراز هویت همراه آن، به یکی از دو نقش اصلی نسبت می دهد. نقش anon به درخواست هایی تعلق می گیرد که فقط با کلید عمومی (Publishable/Anon Key) ارسال شده اند، بدون آنکه کاربر وارد سیستم شده باشد — یعنی هر بازدیدکننده ی معمولی سایت. نقش authenticatedبه درخواست هایی تعلق می گیرد که همراه با یک نشست معتبر ورود(Session Token) ، معمولاً پس از فراخوانی موفق(supabase.auth.signInWithPassword() ، ارسال شده اند — یعنی مدیر سایت پس از ورود موفق.

نکته ی فنی مهمی که در طول توسعه ی این پروژه چندین بار منشأ خطا شد، این بود که این دو نقش مستقل از قصد کاربر، بلکه صرفاً بر اساس وضعیت فعلی مرورگر تعیین می شوند. برای نمونه، وقتی مدیر سایت پس از ورود به پنل مدیریت، برای آزمایش به بخش های عمومی سایت (مانند صفحه ی اصلی یا بلاگ) سر می زد، تمام درخواست های ارسالی از همان مرورگر — از جمله ثبت بازدید صفحه یا ارسال کامنت — با نقش authenticated ارسال می شدند، نه anon. در نتیجه، اگر سیاستی فقط برای نقش anon تعریف شده بود (مثلاً «اجازه ی ثبت بازدید صفحه برای بازدیدکننده ی عادی»)، همان عملیات وقتی مدیر لاگین کرده انجامش می داد، با خطای نقض RLS رد می شد. راه حل این مشکل، تعریف صریح سیاست های مشابه برای هر دو نقش در عملیات هایی بود که باید توسط هر دو گروه کاربر قابل انجام باشند (مانند ثبت بازدید صفحه، ثبت رویداد کلیک، و ارسال کامنت).

۴.۶.۲. مزیت RLS: امنیت در سطح دیتابیس، نه فقط در کد فرانت اند

در معماری هایی که فرانت اند مستقیماً به پایگاه داده متصل می شود، یک اشتباه رایج این تصور است که کافی است بررسی های امنیتی (مانند «آیا کاربر مدیر است») در کد React نوشته شود — برای نمونه، مخفی کردن دکمه ی «حذف مقاله» از بازدیدکننده ی عادی. اما چنین بررسی ای صرفاً یک تجربه ی کاربری بهتر ایجاد می کند، نه امنیت واقعی؛ چون کد جاوااسکریپت اجراشده در مرورگر کاربر، همیشه توسط خود کاربر قابل مشاهده، دستکاری، و دور زدن است. هر بازدیدکننده ای می تواند مستقیماً از طریق کنسول توسعه دهنده ی مرورگر، بدون توجه به هیچ دکمه یا شرط مخفی شده ای، یک درخواست حذف مستقیم به Supabase بفرستد.

Row Level Securityاین مشکل بنیادین را با انتقال منطق کنترل دسترسی از لایه ی فرانت اند به درون خود پایگاه داده حل می کند. وقتی سیاستی مانند «فقط نقش authenticated اجازه ی حذف از جدول posts را دارد» تعریف می شود، PostgreSQLاین قانون را در سطح خود موتور پایگاه داده، برای هر درخواستی — چه از طریق رابط کاربری رسمی سایت بیاید، چه مستقیماً از کنسول مرورگر یا حتی یک ابزار خارجی مانند — Postman به طور یکسان اعمال می کند. به بیان دیگر، امنیت دیگر به «نوشتن درست کد فرانت اند» وابسته نیست، بلکه یک ویژگی ذاتی و غیرقابل دورزدن خود داده است؛ حتی اگر توسعه دهنده فراموش کند یک دکمه را مخفی کند، تا وقتی سیاست RLS درستی تعریف شده باشد، آن عملیات غیرمجاز از سمت پایگاه داده رد خواهد شد. این مزیت دقیقاً همان چیزی است که در فصل دوم این سند، به عنوان دلیل اصلی امکان پذیری معماری «فرانت اند مستقیم متصل به «Backend-as-a-Service بدون نیاز به یک لایه ی بک اند واسط، مطرح شد.

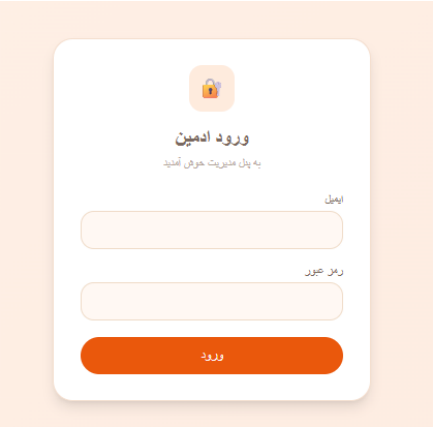
فصل پنجم: رابط کاربری

۵.۱. هویت بصری و پالت رنگی

طراحی بصری صفحات جدید این پروژه (بلاگ، فرم ورود مدیر، پنل مدیریت) بر پایه ی یک اصل راهنما شکل گرفت: این صفحات نباید هیچ گسست بصری ای با سایت اصلی ایجاد کنند، حتی اگر از نظر کارکردی کاملاً متفاوت باشند (یک داشبورد آماری در مقابل یک صفحه ی معرفی خدمات). برای دستیابی به این هدف، پالت رنگی سایت اصلی به طور کامل استخراج و در تمام کامپوننت های جدید بازتولید شد: پس زمینه ای با گرادینت نرم از کرم به هلویی (#FFF8F3 تا #FFF3EA)، رنگ متن اصلی قهوه ای گرم (#A69607) که همان رنگ لوگوی سایت است، و رنگ تاکیدی نارنجی (#EE7540 و bg-orange-600) که در تمام دکمه های عملیاتی — از «ورود» در فرم لاگین تا «ذخیره» در فرم مقاله — به طور یکسان به کار رفت.

فراتر از رنگ، الگوهای شکلی سایت اصلی نیز حفظ شدند: گوشه های به شدت گرد (rounded-2xl و rounded-3xl به جای گردی معمول و کوچک)، سایه های بسیار ملایم به جای خطوط مرزی تیز، و افکت درخشش نارنجی هنگام هاور روی دکمه ها (hover:shadow hover:shadow-orange-500). حتی جزئیاتی مانند رنگ بندی وضعیت ها (کارت های سبز کم رنگ برای «منتشرشده» یا «تاییدشده»، زرد کم رنگ برای «در انتظار») به جای رنگ های اشباع و پررنگ معمول رابط های مدیریتی، با همان درجه از ملایمت پالت اصلی سایت هماهنگ شدند. نتیجه

ی این یکپارچگی این بود که یک بازدیدکننده که از صفحه ی اصلی سایت به بلاگ حرکت می کند، یا یک مدیر که از پنل مدیریتی خارج و وارد بخش عمومی سایت می شود، هیچ لحظه ای احساس نمی کند وارد یک محصول یا پروژه ی جداگانه شده است.

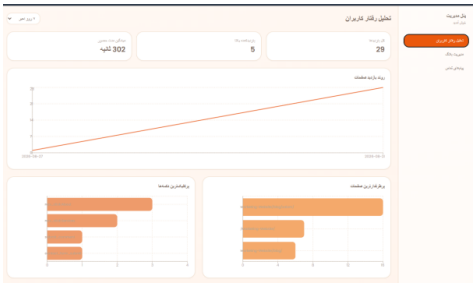


۵.۲. ساختار پنل مدیریت

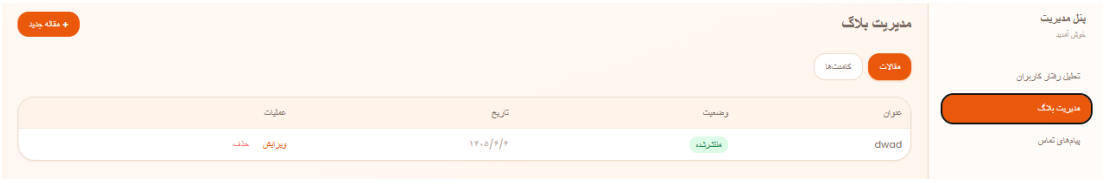
پنل مدیریت با یک الگوی چیدمان ثابت و آشنا طراحی شده: یک سایدبار عمودی سمت راست (متناسب با جهت راست به چپ زبان فارسی) که همیشه در دسترس است، و یک ناحیه ی محتوای اصلی در سمت چپ که بر اساس مسیر فعال (آنالیتیکس، بلاگ، یا پیام ها) تغییر می کند. این ساختار — که در فصل سوم این سند به عنوان یکی از کاربردهای مسیرهای تودرتوی React Router توضیح داده شد — از نظر تجربه ی کاربری چند مزیت مستقیم دارد: نخست، مدیر همیشه می داند در کدام بخش از سیستم قرار دارد، چون لینک فعال در سایدبار با رنگ نارنجی پررنگ (`bg-orange-۶۰۰`) از بقیه ی لینک ها متمایز می شود. دوم، جابه جایی بین بخش ها بدون بارگذاری مجدد کل صفحه انجام می شود، چون فقط ناحیه ی `Outlet` </> تغییر می کند نه کل چیدمان. سوم، عناصر ثابت و پرکاربرد — به ویژه دکمه ی «خروج» — همیشه در یک موقعیت قابل پیش بینی (پایین سایدبار) در دسترس اند، مستقل از اینکه مدیر در کدام زیربخش پنل باشد.

درون این ساختار کلی، هر صفحه از الگوهای یکسانی برای نمایش داده استفاده می کند تا حس یکپارچگی حتی در سطح جزئیات حفظ شود: کارت های آماری (`StatCard`) در داشبورد آنالیتیکس، سطرهای جدول در مدیریت بلاگ، و کارت های پیام در صندوق پیام ها، همگی از یک زبان بصری مشترک (کارت سفید با گوشه ی گرد، حاشیه ی کرم رنگ ملایم، و سایه ی سبک) پیروی می کنند، به طوری که مدیر بدون نیاز به یادگیری مجدد الگوهای بصری در هر بخش تازه، بلافاصله می داند چگونه با یک عنصر جدید تعامل کند.

	UID	Display name	Email	Phone
	6dd508cf-5fa6-45f3-bc63-d1d93d1f0152	-	ax@usc.ir	-
	01d99744-e96d-4a56-b7ff-abbb3e9786cd	-	usc@iran.ir	-







۵.۳. واکنش گرایی (Responsive Design)

طراحی واکنش گرا در این پروژه به طور کامل بر پایه ی کلاس های Breakpoint داخلی Tailwind CSS (sm:، md:، lg:) پیاده سازی شده، بدون نیاز به نوشتن هیچ Media Query سفارشی. رویکرد کلی، طراحی «موبایل-اول» (Mobile-First) است: استایل پایه ی هر عنصر برای کوچک ترین اندازه ی صفحه نوشته می شود، و کلاس های پیشونددار برای اندازه های بزرگ تر، چیدمان را تدریجاً غنی تر می کنند. برای نمونه، شبکه ی کارت های مقالات در صفحه ی بلاگ با `grid-cols-۱` شروع می شود (یک ستون در موبایل) و با `md:grid-cols-۲` به دو ستون در صفحه نمایش های متوسط به بالا تبدیل می شود؛ به همین ترتیب، کارت های آماری داشبورد آنالیتیکس از یک ستون در موبایل به سه ستون در `sm:grid-cols-۳` می رسند.

واکنش گرایی پنل مدیریت با چالش بیشتری همراه بود، چون سایدبار ثابت عرض ۶۴ واحدی (`w-٦٤`) در صفحه نمایش های کوچک، سهم نامتناسبی از فضای محدود را اشغال می کند. با این حال، از آن جا که کاربرد اصلی و پیش بینی شده ی پنل مدیریت، مدیریت محتوا از طریق دسکتاپ است (نه یک وظیفه ی معمول موبایلی)، اولویت طراحی بر خوانایی و کارایی در صفحه نمایش های بزرگ تر گذاشته شد، در حالی که بخش های عمومی سایت — که مستقیماً توسط طیف گسترده تری از بازدیدکنندگان، از جمله کاربران موبایل، دیده می شوند — با دقت بیشتری برای تمام اندازه های صفحه بهینه شدند؛ برای نمونه، منوی موبایل سایت اصلی (که پیش از این پروژه نیز وجود داشت) با افزودن لینک «بلاگ» به همان الگوی منوی کشویی همبرگری موجود، بدون نیاز به بازطراحی، گسترش یافت.



فصل ششم: ماژول بلاگ

۶.۱ .مدیریت مقالات (افزودن/ویرایش/حذف)

عملیات مدیریت مقالات در پنل ادمین حول دو کامپوننت اصلی سازمان دهی شده است: BlogAdmin.jsx که فهرست تمام مقالات را نمایش می دهد، و BlogEditor.jsx که به صورت مشترک برای عملیات افزودن و ویرایش به کار می رود. این کامپوننت واحد، با بررسی وجود یا عدم وجود پارامتر id در آدرس URL (از طریق شرط isNew = !id || id === "new")، تشخیص می دهد کاربر در حال ساخت مقاله ی تازه است یا ویرایش یکی از مقالات موجود؛ در حالت ویرایش، داده ی فعلی مقاله از پایگاه داده خوانده و در فرم پیش بارگذاری می شود، و در حالت افزودن، فرم با مقادیر خالی و وضعیت پیش فرض draft آغاز می شود. این طراحی مشترک، از تکرار کد میان دو حالت مفهوماً مشابه (هردو در نهایت یک عملیات ذخیره ی insert یا update روی جدول posts هستند) جلوگیری کرد.

عملیات حذف در BlogAdmin.jsx پیاده سازی شده و پیش از اجرا، از طریق یک تاییدیه ی مرورگری ساده (confirm()) از مدیر تاییدیه می گیرد تا از حذف اتفاقی جلوگیری شود. با توجه به قید on delete cascade تعریف شده در سطح پایگاه داده (شرح فصل ۴.۵)، حذف یک مقاله به طور خودکار تمام کامنت های وابسته به آن را نیز حذف می کند، بدون نیاز به نوشتن منطق حذف جداگانه برای کامنت ها در سمت فرانت اند.

۶.۲. سیستم Slug برای لینک خوانا

در نسخه ی اولیه ی این ماژول، لینک هر مقاله از ترکیب یک نسخه ی ساده شده ی عنوان و یک برچسب زمانی تصادفی (slugify(title)-Date.now()) ساخته می شد، که نتیجه اش آدرس هایی غیرخوانا و نامناسب برای سئو بود. در تکرار بعدی، این سیستم به یک فیلد Slug مستقل و قابل ویرایش دستی ارتقا یافت: با تایپ عنوان، تابع slugify() به طور خودکار پیش نویسی از لینک می سازد (حذف نویسه های غیرمجاز، تبدیل فاصله به خط تیره)، اما تا زمانی که کاربر خودش این فیلد را دستی ویرایش نکرده باشد (slugEditedManually)، این همگام سازی خودکار ادامه می یابد؛ به محض اولین ویرایش دستی، همگام سازی خودکار متوقف می شود تا تغییرات عنوان بعدی، لینکی را که مدیر عمداً انتخاب کرده، بازنویسی نکند.

از آن جا که ستون slug در پایگاه داده با قید unique تعریف شده، تلاش برای ذخیره ی یک لینک تکراری با خطای کد ۲۳۵۰۵ از سمت PostgreSQL مواجه می شود؛ این خطا به طور خاص در فرم شناسایی و با پیامی واضح («این لینک قبلاً برای یک مقاله ی دیگر استفاده شده») به مدیر نمایش داده می شود، به جای یک پیام خطای فنی و مبهم.

۶.۳. سئوی مقاله (متا تایتل، متا دیسکریپشن، ALT تصویر)

برای پشتیبانی از عناصر پایه ای سئو در سطح هر مقاله، سه فیلد اختیاری به فرم افزوده شد: متا تایتل (عنوانی که در نتایج موتور جستجو و تب مرورگر نمایش داده می شود، مستقل از عنوان اصلی مقاله)، متا دیسکریپشن (توضیح کوتاهی که زیر لینک در نتایج جستجو ظاهر می شود)، و متن جایگزین تصویر کاور (ALT). اعمال عملی این مقادیر در کامپوننت BlogPost.jsx انجام می شود: یک useEffect مجزا، هنگام بارگذاری مقاله، مقدار document.title صفحه را به روزرسانی می کند و یک تگ <meta name="description"> را در <head> سند می سازد یا ویرایش می کند؛ و مهم تر، هنگام خروج کاربر از صفحه (Cleanup Function)، این تغییرات به مقدار قبلی خود بازمی گردند تا از باقی ماندن متا اطلاعات یک مقاله روی صفحات دیگر سایت جلوگیری شود.

۶.۴. نشانه گذاری متن سبک (تیتر، لینک)

به جای افزودن یک ویرایشگر متن گرافیکی کامل (WYSIWYG Editor) که پیچیدگی و وزن قابل توجهی به پروژه اضافه می کرد، برای این پروژه یک راه حل سبک تر انتخاب شد: نویسنده مستقیماً در یک <textarea> ساده می نویسد و از یک نشانه گذاری متنی محدود (مشابه Markdown سبک شده) برای قالب بندی استفاده می کند — علامت #، ##، و ### در ابتدای یک خط به ترتیب به تیتر بزرگ، متوسط و کوچک تبدیل می شوند، و الگوی [متن](آدرس) به یک لینک قابل کلیک. تبدیل این نشانه ها به عناصر واقعی React، توسط یک تابع مستقل به نام renderContent() (در فایل src/lib/renderContent.jsx) انجام می شود که متن را ابتدا بر اساس خطوط خالی به پاراگراف ها تقسیم می کند، سپس هر بلوک را بر اساس نشانه ی ابتدایی اش به تگ متناظر (h۱، h۲، h۳، یا p) نگاشت می کند، و در نهایت با یک عبارت باقاعده (Regular Expression) الگوهای لینک درون هر بلوک را جایگزین می کند. این رویکرد، پیچیدگی پیاده سازی را به شدت کاهش داد و در عین حال، امکان ساخت مقالاتی با ساختار سلسله مراتبی و لینک دار را بدون وابستگی به یک کتابخانه ی خارجی فراهم کرد.

۶.۵. سیستم کامنت و تایید مدیر

سیستم کامنت گذاری بر پایه ی یک اصل ساده طراحی شده: هر کامنتی که کاربر عمومی ثبت می کند، بلافاصله عمومی نمی شود. هنگام ارسال فرم کامنت در BlogPost.jsx، یک ردیف جدید با وضعیت پیش فرض pending در جدول comments درج می شود، و کاربر پیامی مبنی بر «نظر شما پس از تایید نمایش داده می شود» دریافت می کند. این کامنت تا زمانی که مدیر آن را از تب «کامنت ها» در BlogAdmin.jsx بررسی و به وضعیت approved تغییر ندهد، در بخش عمومی مقاله نمایش داده نمی شود (چرا که کوئری نمایش کامنت ها در BlogPost.jsx صراحتاً با شرط eq("status", "approved") فیلتر شده است).

برای کمک به مدیر در پیگیری کامنت های در انتظار، یک نشانگر عددی (Badge) قرمزرنگ کنار تب «کامنت ها» تعداد کامنت های با وضعیت pending را نمایش می دهد. مدیر برای هر کامنت سه اقدام در اختیار دارد: تایید (approved)، رد (rejected — که کامنت را بدون حذف کامل، از نمایش عمومی خارج نگه می دارد)، یا حذف کامل از پایگاه داده. این جداسازی میان «رد کردن» و «حذف کردن» عمداً انجام شده تا در صورت نیاز به بازبینی بعدی یا بررسی الگوهای اسپم، سابقه ی کامنت های ردشده در پایگاه داده باقی بماند.

فصل هفتم: ماژول تحلیل رفتار کاربر

۷.۱. ردیابی بازدید صفحه

هسته ی ردیابی بازدید صفحه در تابع trackPageView() (فایل src/lib/analytics.js) پیاده سازی شده و از طریق یک Custom Hook به نام usePageTracking() به طور خودکار در سراسر اپلیکیشن فعال می شود. این هوک از useLocation() کتابخانه ی React Router برای تشخیص هر تغییر در مسیر URL استفاده می کند؛ به این معنا که با هر بار جابه جایی کاربر بین صفحات سایت (چه از طریق کلیک روی لینک، چه از طریق دکمه ی Back مرورگر)، این هوک به طور خودکار اجرا می شود، بدون نیاز به فراخوانی دستی در هر کامپوننت صفحه به طور جداگانه.

پیش از ثبت هر بازدید، تابع getSessionId() بررسی می کند آیا برای مرورگر فعلی، شناسه ی نشستی از قبل در sessionStorage ذخیره شده یا خیر؛ در صورت نبود، یک شناسه ی یکتا با crypto.randomUUID() تولید و هم در sessionStorage و هم در جدول sessions پایگاه داده ثبت می شود. یک تصمیم طراحی عمدی در این بخش این بود که ردیابی صفحات مسیرهای زیرمجموعه ی /admin به طور کامل حذف شد (با یک شرط ساده در ابتدای هوک: if (location.pathname.includes("/admin")) return;)، چرا که هدف این ماژول، شناخت رفتار بازدیدکنندگان سایت، نه فعالیت های داخلی خود مدیر در پنل مدیریت است؛ بدون این استثنا، آمار داشبورد با رفت وآمدهای خود مدیر در حین کار روزمره آلوده می شد.

۷.۲. محاسبه ی مدت حضور

محاسبه ی مدت حضور در هر صفحه بر پایه ی یک الگوی «شروع و پایان» طراحی شده است. تابع trackPageView() هنگام فراخوانی، زمان دقیق ورود به صفحه را در متغیر startedAt ذخیره می کند و بلافاصله یک ردیف تازه در جدول page_views درج می کند (بدون مقدار اولیه برای duration_seconds)؛ سپس یک تابع به نام finish بازمی گرداند که مسئول محاسبه ی نهایی مدت حضور است. این تابع finish در دو نقطه فراخوانی می شود: نخست، در Cleanup Function هوک usePageTracking که هنگام خروج کاربر از یک مسیر و ورود به مسیر بعدی به طور خودکار توسط React اجرا می شود؛ و دوم، در رویداد beforeunload مرورگر، برای پوشش موردی که کاربر مستقیماً تب یا پنجره ی مرورگر را می بندد (که در آن حالت، چرخه ی عادی Unmount کامپوننت React رخ نمی دهد).

در مسیر توسعه ی این بخش، یک اشکال ظریف اما مهم شناسایی و رفع شد: از آن جا که هم عملیات «ثبت اولیه ی بازدید» و هم عملیات «به روزرسانی مدت حضور» هر دو درخواست های شبکه ای غیرهمزمان (Asynchronous) هستند، در

شرایطی که کاربر بسیار سریع از یک صفحه خارج می شد، درخواست به روزرسانی می توانست پیش از تکمیل درخواست ثبت اولیه به پایگاه داده برسد و در نتیجه هیچ ردیفی برای به روزرسانی پیدا نمی کرد (شرایطی موسوم به Race Condition). راه حل نهایی، نگه داشتن Promise عملیات درج اولیه در متغیری به نام insertPromise و اطمینان از تکمیل آن (insertPromise.then(...)) پیش از اجرای عملیات به روزرسانی بود؛ این تغییر، اجرای دو عملیات را به طور تضمینی به ترتیب صحیح درآورد، بدون نیاز به تاخیر مصنوعی یا پیچیده کردن منطق برنامه.

۷.۳. ردیابی کلیک دکمه ها/رویدادها

ردیابی کلیک، برخلاف ردیابی صفحه که به طور کاملاً خودکار و سراسری عمل می کند، به صورت دستی و انتخابی روی عناصر مشخصی از رابط کاربری اعمال شده است. تابع trackEvent(eventName, path) در فایل analytics.js یک ردیف ساده در جدول events ثبت می کند و مستقیماً در رویداد onClick عناصر هدف فراخوانی می شود. این فراخوانی در دو سطح از پروژه انجام شده: نخست، در سطح کامپوننت مشترک CtaButton.jsx (که در چندین نقطه از سایت، از جمله دکمه های «Get Started»، استفاده می شود)، به طوری که هر استفاده از این کامپوننت به طور خودکار ردیابی می شود بدون نیاز به کد تکراری در هر نقطه ی استفاده. دوم، به صورت مورد به مورد روی لینک های خاص نوار ناوبری (About، Services، Portfolio) که چون از تگ <a> ساده استفاده می کردند نه کامپوننت مشترک، نیازمند افزودن onClick={() => trackEvent("nav_click:About")} به طور جداگانه بودند.

نام گذاری رویدادها از یک قرارداد ساده با پیشوند معنادار پیروی می کند (برای نمونه cta_click:Get Started یا nav_click:About)، که این ساختار در فصل بعد (داشبورد) امکان دسته بندی و شمارش رویدادها بر اساس نام دقیقشان را فراهم می کند. کامنت های ارسالی در بلاگ نیز به عنوان یک رویداد قابل ردیابی (submit_comment) ثبت می شوند، که نشان می دهد این سیستم صرفاً به کلیک دکمه محدود نیست، بلکه هر اقدام معنادار کاربر را می توان به همین شیوه ثبت کرد.

۷.۴. داشبورد و نمودارها

کامپوننت AnalyticsDashboard.jsx مصرف کننده ی نهایی تمام داده ی ثبت شده توسط سه بخش قبل است. این داشبورد با یک منوی کشویی، امکان انتخاب بازه ی زمانی (امروز، هفت روز اخیر، سی روز اخیر) را در اختیار مدیر قرار می دهد؛ با هر تغییر این بازه، یک useEffect داده ی خام جدول های page_views و events را از پایگاه داده (فیلترشده با شرط gte("created_at", since)) بازخوانی می کند.

پردازش این داده ی خام به آمار قابل نمایش، به طور کامل در سمت مرورگر و با استفاده از هوک useMemo انجام می شود — رویکردی که از اجرای کوئری های تجمیعی پیچیده در سطح SQL خودداری کرده و منطق محاسباتی را در جاوااسکریپت ساده و قابل فهم نگه می دارد. سه محاسبه ی اصلی در این بخش انجام می شود: شمارش کل بازدیدها و شمارش تعداد session_id های یکتا (با استفاده از شیء Set جاوااسکریپت) برای بازدیدکننده ی یکتا؛ میانگین گیری از

مقادیر معتبر duration_seconds (با فیلتر کردن مقادیر null یا صفر) برای میانگین مدت حضور؛ و گروه بندی رکوردها بر اساس تاریخ یا مسیر صفحه (با استفاده از یک شیء نگاشت ساده) برای ساخت داده ی مناسب نمودار.

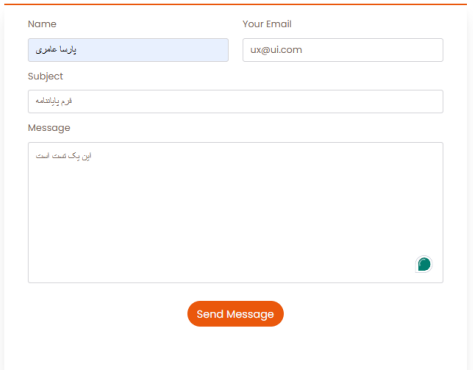
نتیجه ی این محاسبات با کتابخانه ی Recharts (شرح فصل ۳.۸) به صورت سه نمودار مجزا نمایش داده می شود: یک نمودار خطی برای روند بازدید روزانه، و دو نمودار میله ای افقی برای پرطرفدارترین صفحات و پرکلیک ترین دکمه ها، که هر دو به هشت مورد برتر محدود شده اند تا خوانایی نمودار در فضای محدود صفحه حفظ شود.

فصل هشتم: ماژول فرم تماس

۸.۱. ثبت پیام در دیتابیس

فرم تماس این پروژه از پیش، در نسخه ی اولیه ی سایت اصلی، از کتابخانه ی React Hook Form برای مدیریت فیلدها استفاده می کرد (شرح فصل ۳.۵)، اما تابع onSubmit آن صرفاً یک اعلان موفقیت نمایش می داد بدون آنکه داده ای در جایی ذخیره شود. در این پروژه، این تابع بازنویسی شد تا به جای رفتار نمایشی صرف، عملیات واقعی ذخیره سازی را انجام دهد: با فراخوانی supabase.from("messages").insert({...})، چهار فیلد فرم (نام، ایمیل، موضوع، متن پیام) مستقیماً به عنوان یک ردیف تازه در جدول messages درج می شوند.

از آن جا که این عملیات یک درخواست شبکه ای غیرهمزمان است، یک وضعیت محلی به نام sending به کامپوننت افزوده شد که در بازه ی زمانی میان ارسال درخواست و دریافت پاسخ، دکمه ی ارسال را غیرفعال می کند؛ این تصمیم طراحی به طور خاص از یک مشکل رایج در فرم های وب جلوگیری می کند: ارسال چندباره ی یک پیام در نتیجه ی چند بار کلیک متوالی کاربر روی دکمه، پیش از دریافت بازخورد اول. در صورت بروز خطا در عملیات درج (برای نمونه به دلیل قطع شبکه یا رد شدن سیاست RLS)، خطا در کنسول مرورگر ثبت می شود و یک اعلان (Toast) قرمزرنگ با کتابخانه ی React Toastify به کاربر نمایش داده می شود؛ در صورت موفقیت، یک اعلان سبزرنگ ظاهر می شود و فرم با متد reset() کتابخانه ی React Hook Form به طور کامل پاک سازی می شود تا آماده ی یک پیام تازه باشد. ارسال موفق هر پیام، هم زمان به عنوان یک رویداد قابل ردیابی (contact_form_submit، شرح فصل ۷.۳) نیز ثبت می شود، که امکان سنجش نرخ تبدیل بازدیدکننده به تماس گیرنده را در داشبورد آنالیتیکس فراهم می کند.

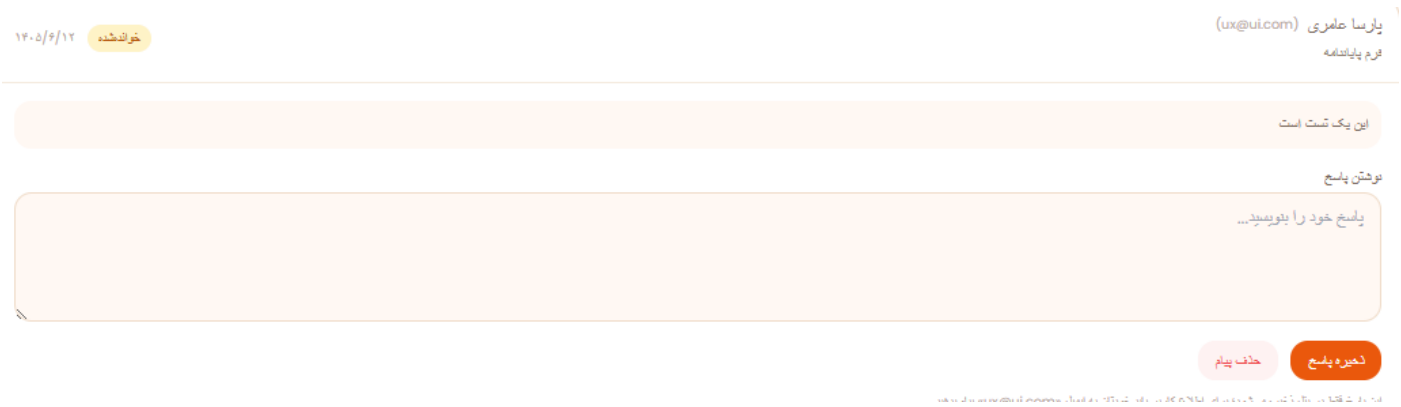




۸.۲. صندوق پیام مدیریتی

کامپوننت MessagesAdmin.jsx تمام پیام های ثبت شده در جدول messages را، مرتب شده بر اساس جدیدترین تاریخ، در قالب فهرستی از کارت های تاشو (Accordion-style) نمایش می دهد. هر کارت در حالت بسته صرفاً اطلاعات خلاصه (نام، ایمیل، موضوع، وضعیت، و تاریخ) را نشان می دهد؛ با کلیک روی هر پیام، آن کارت باز می شود و متن کامل پیام، همراه با پاسخ قبلی مدیر (در صورت وجود) و یک فرم پاسخ دهی، نمایان می شود.

یک رفتار خودکار در این کامپوننت طراحی شده: به محض باز کردن یک پیام با وضعیت unread، بلافاصله و بدون نیاز به اقدام جداگانه ی مدیر، وضعیت آن پیام در پایگاه داده به read تغییر می کند (await supabase.from("messages").update({ status: "read" })). این الگو مشابه رفتار متداول در برنامه های ایمیل و پیام رسان است و از مدیر نیاز به یک اقدام دستی جداگانه («علامت گذاری به عنوان خوانده شده») را حذف می کند. برای کمک به مدیر در شناسایی سریع پیام های تازه، تعداد پیام های با وضعیت unread در بالای صفحه به صورت یک نشانگر برجسته نمایش داده می شود، مشابه الگوی مشابهی که برای کامنت های در انتظار تایید در ماژول بلاگ به کار رفته است (فصل ۶.۵) — نشان دهنده ی یک الگوی طراحی تکرارشونده و آگاهانه در سراسر پنل مدیریت این پروژه.



۸.۳. پاسخ دهی مدیر

برای هر پیام باز، یک ناحیه ی متنی (Textarea) برای نگارش یا ویرایش پاسخ در اختیار مدیر قرار دارد؛ با کلیک روی دکمه ی «ذخیره پاسخ»، سه ستون هم زمان در پایگاه داده به روزرسانی می شوند: متن پاسخ (admin_reply)، زمان ثبت پاسخ (replied_at، با مقدار new Date().toISOString())، و وضعیت پیام که به replied تغییر می یابد. طراحی این بخش امکان ویرایش مجدد یک پاسخ قبلاً ثبت شده را نیز پشتیبانی می کند؛ اگر پیامی از پیش پاسخ داده شده باشد، متن پاسخ موجود در همان Textarea بارگذاری می شود و برچسب دکمه از «نوشتن پاسخ» به «ویرایش پاسخ» تغییر می کند.



نکته ی مهمی که باید به صراحت در طراحی این بخش در نظر گرفته شود، محدودیت ذاتی معماری این پروژه است: چون هیچ سرویس ارسال ایمیل (مانند Resend یا SendGrid) در این نسخه به کار نرفته، پاسخ ثبت شده توسط مدیر صرفاً درون پایگاه داده و پنل مدیریت ذخیره می شود و هیچ اعلان یا ایمیلی به طور خودکار برای کاربری که پیام را ارسال کرده به راه نمی افتد. برای شفافیت این محدودیت نزد مدیر، یک یادداشت کوچک زیر فرم پاسخ («این پاسخ فقط در پنل ذخیره می شود؛ برای اطلاع کاربر باید خودتان به ایمیل کاربر پیام بدهید») درج شده است. این محدودیت به عنوان یکی از مسیرهای مشخص توسعه ی آینده در فصل پایانی این سند دوباره مطرح خواهد شد.

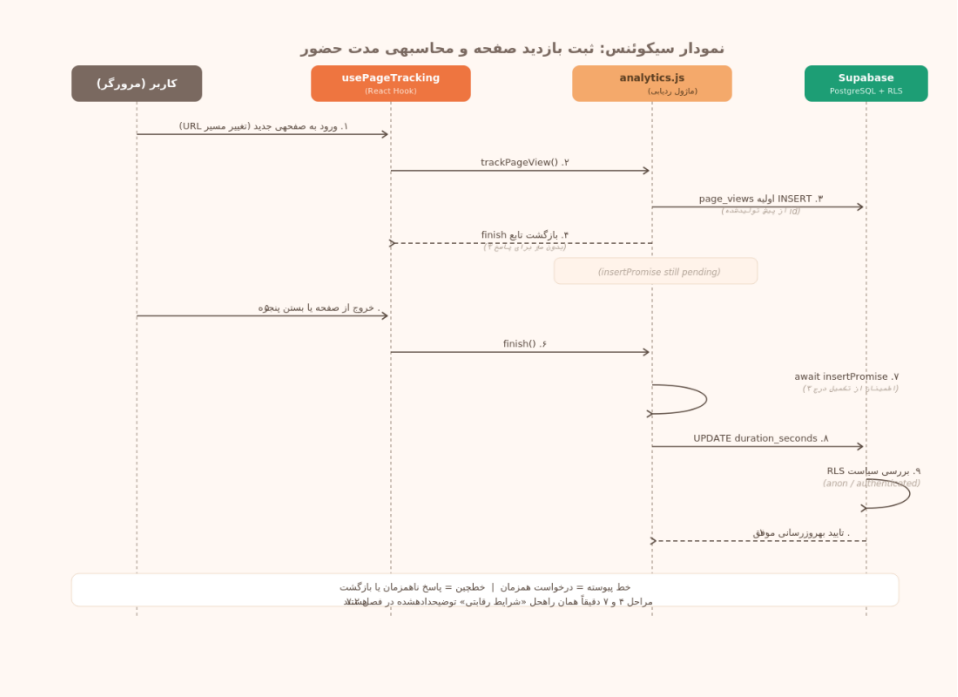
فصل نهم: مسیرها و امنیت دسترسی

۹.۱ جدول کامل مسیرها

تمام مسیرهای این پروژه در یک فایل مرکزی به نام utils/Routings.jsx تعریف شده اند. جدول زیر فهرست کامل این مسیرها، به همراه کامپوننت متناظر و سطح دسترسی هر یک را نشان می دهد:

جدول کامل مسیرهای پروژه (Routes)

مسیر	کامپوننت	نوع دسترسی
`/blog`	BlogList	عمومی
`/blog/:slug`	BlogPost	عمومی
`/admin/login`	AdminLogin	عمومی
`/admin`	AdminLayout (والد)	محافظت‌شده
`/admin` (index)	تغییر مسیر خودکار به analytics	محافظت‌شده
`/admin/analytics`	AnalyticsDashboard	محافظت‌شده
`/admin/blog`	BlogAdmin	محافظت‌شده
`/admin/blog/new`	BlogEditor (افزودن)	محافظت‌شده
`/admin/blog/:id`	BlogEditor (ویرایش)	محافظت‌شده
`/admin/messages`	MessagesAdmin	محافظت‌شده



یک نکته ی ساختاری قابل توجه در این جدول، تفاوت پارامتر مسیر میان صفحه ی عمومی و صفحه ی مدیریتی مقاله است: مسیر عمومی از slug (لینک خوانا، مانند my-first-post) استفاده می کند، در حالی که مسیر ویرایش در پنل مدیریت از id (شناسه ی داخلی uuid) استفاده می کند. این تفاوت عمدی است — بازدیدکننده ی عمومی با آدرسی خوانا و سئوپسند سروکار دارد، در حالی که پنل مدیریت، که کاربرش همیشه از طریق کلیک روی دکمه ی «ویرایش» (نه تایپ مستقیم آدرس) وارد این صفحه می شود، از شناسه ی داخلی و تضمیناً یکتای پایگاه داده استفاده می کند.

همچنین باید توجه داشت که تمام این مسیرها، به دلیل دیپلوی پروژه روی GitHub Pages با یک مسیر پایه ی غیرریشه ('base: '/Marketing-Website/' در تنظیمات Vite)، در عمل با این پیشوند ترکیب می شوند (برای نمونه، مسیر واقعی و کامل بلاگ در محیط اجرا Marketing-Website/blog/ است).

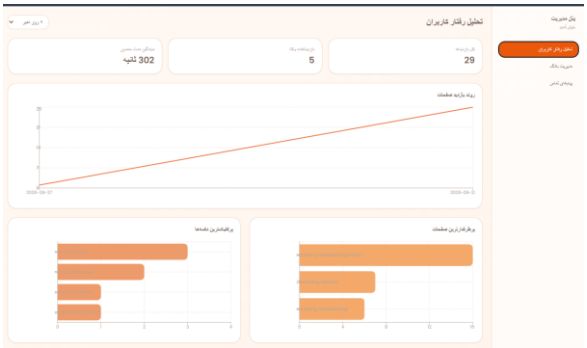
۹.۲. محافظت از پنل ادمین (ProtectedRoute)

محافظت از تمام مسیرهای زیرمجموعه ی /admin (به جز خود صفحه ی ورود) از طریق یک کامپوننت واسط به نام ProtectedRoute.jsx پیاده سازی شده که با یک الگوی طراحی رایج در React موسوم به Higher-Order Component غیرمستقیم عمل می کند: به جای اینکه هر صفحه ی مدیریتی به طور جداگانه مسئول بررسی وضعیت ورود باشد، این بررسی یک بار و در بالاترین سطح ممکن از سلسله مراتب مسیرهای مدیریتی انجام می شود. در فایل Routings.jsx، کامپوننت AdminLayout (که شامل سایدبار و ناحیه ی محتوای مشترک تمام صفحات مدیریتی است) درون ProtectedRoute قرار گرفته؛ از آن جا که تمام صفحات مدیریتی دیگر (آنالیتیکس، بلاگ، پیام ها) به عنوان مسیر فرزند همین AdminLayout تعریف شده اند، این یک لایه ی محافظتی، خودکار تمام آن ها را پوشش می دهد.

از نظر منطق داخلی، ProtectedRoute وضعیت ورود را از AuthContext (شرح فصل ۲.۱ و بخش های مرتبط) از طریق هوک useAuth() می خواند و بر اساس سه حالت ممکن رفتار متفاوتی نشان می دهد. حالت نخست، در حال بررسی (isLoading)، زمانی رخ می دهد که AuthContext هنوز از Supabase Auth پاسخ نگرفته که آیا نشست معتبری

برای این مرورگر وجود دارد یا نه — در این حالت، یک پیام ساده ی «در حال بارگذاری» نمایش داده می شود تا از یک تصمیم زودهنگام و اشتباه (مثلاً هدایت یک مدیر واقعاً لاگین کرده به صفحه ی ورود، صرفاً به این دلیل که پاسخ هنوز نرسیده) جلوگیری شود. حالت دوم، عدم ورود (!isAdmin)، زمانی رخ می دهد که بررسی کامل شده و مشخص شده کاربر نشست معتبری ندارد؛ در این حالت، کامپوننت <Navigate> از React Router کاربر را به /admin/login هدایت می کند، و هم زمان مسیر اصلی که کاربر قصد داشت به آن برسد را در state.from ذخیره می کند تا پس از ورود موفق، به جای صفحه ی پیش فرض داشبورد، مستقیماً به همان مقصد اولیه بازگردانده شود. حالت سوم، ورود معتبر (isAdmin === true)، به سادگی محتوای فرزند (children) را بدون هیچ مانعی رندر می کند.

آنچه این لایه ی محافظتی را از یک بررسی صرفاً تجربه ی کاربری (مانند مخفی کردن یک لینک در منو) متمایز می کند، این است که این محافظت صرفاً یک لایه ی مکمل است، نه تنها خط دفاعی؛ همان طور که در فصل ۴.۶.۲ توضیح داده شد، حتی اگر کاربری با دور زدن این کامپوننت React (برای نمونه با دستکاری مستقیم درخواست های شبکه) بتواند وارد یک مسیر مدیریتی شود، سیاست های Row Level Security در سطح پایگاه داده، مانع از خواندن یا نوشتن هرگونه داده ای می شوند که او مجاز به دسترسی به آن نیست. به بیان دیگر، ProtectedRoute تجربه ی کاربری را کنترل می کند («این کاربر نباید این صفحه را ببیند»)، در حالی که RLS امنیت واقعی داده را تضمین می کند («این کاربر نمی تواند این داده را بخواند، فارغ از اینکه چه صفحه ای می بیند») — دو لایه ی مستقل که با هم یک سیستم دفاع در عمق (Defense in Depth) تشکیل می دهند.



فصل دهم: جمع بندی و پیشنهادات توسعه ی آینده

۱۰.۱. محدودیت های فعلی

با وجود پیاده سازی کامل و کارکردی هر سه ماژول این پروژه، محدوده ی نسخه ی فعلی به طور آگاهانه از برخی قابلیت ها صرف نظر کرده که در ادامه به مهم ترین آن ها پرداخته می شود.

نبود ارسال ایمیل خودکار. همان طور که در فصل هشتم اشاره شد، پاسخ مدیر به پیام های تماس، و همچنین تایید نظرات کامنت، هیچ کدام هیچ اعلانی برای کاربر نهایی ارسال نمی کنند. کاربری که پیامی فرستاده، هیچ راهی برای اطلاع از پاسخ مدیر ندارد جز اینکه خودش با روش دیگری (تماس تلفنی، ایمیل مستقل) پیگیری کند. این محدودیت، نتیجه ی مستقیم معماری Backend-as-a-Service این پروژه است: Supabase به خودی خود سرویس ارسال ایمیل ندارد، و افزودن این قابلیت نیازمند یکپارچه سازی یک سرویس شخص ثالث (مانند Resend یا SendGrid) از طریق یک Supabase

Edge Function است — گامی که عمداً خارج از محدوده ی این نسخه نگه داشته شد تا تمرکز پروژه بر تحویل سریع سه فیچر اصلی حفظ شود.

نبود Rate Limiting و مکانیزم ضد اسپم. سه نقطه ی ورودی عمومی این پروژه — فرم ثبت کامنت، فرم تماس، و خود سیستم ردیابی رفتار کاربر — هیچ کدام هیچ محدودیتی روی تعداد درخواست های ارسالی از یک منبع در بازه ی زمانی مشخص ندارند. از منظر فنی، هر بازدیدکننده (یا هر اسکریپت خودکار) می تواند به طور نامحدود کامنت جعلی، پیام تماس، یا حتی داده ی ردیابی رفتار جعلی به پایگاه داده ارسال کند، که هم می تواند داده ی آماری داشبورد آنالیتیکس را مخدوش کند و هم صندوق پیام و بخش نظرات را با محتوای نامرتبط پر کند. تنها خط دفاعی فعلی در برابر این ریسک، فرآیند تایید دستی کامنت توسط مدیر است (فصل ۶.۵) که یک راه حل واکنشی است، نه پیشگیرانه.

نبود ادیتور متن گرافیکی. همان طور که در فصل ۶.۴ توضیح داده شد، نگارش مقاله متکی به یک نشانه گذاری متنی ساده است، نه یک ویرایشگر WYSIWYG با دکمه های فرمت دهی بصری (Bold، لیست، درج تصویر داخل متن). این محدودیت یک تصمیم آگاهانه برای ساده نگه داشتن پیاده سازی بود، اما تجربه ی نگارش را برای نویسندگانی که با نحو Markdown آشنا نیستند، دشوارتر می کند.

نبود سطوح دسترسی چندگانه. سیستم احراز هویت فعلی تنها یک نوع کاربر ممتاز (مدیر) را می شناسد. امکان تعریف نقش های میانی — برای نمونه، «نویسنده» که فقط بتواند مقاله بنویسد اما به صندوق پیام یا داشبورد آنالیتیکس دسترسی نداشته باشد — در این نسخه وجود ندارد.

۱۰.۲. پیشنهادهای توسعه

بر پایه ی محدودیت های فوق، مسیرهای زیر به عنوان گام های منطقی توسعه ی آینده ی این پروژه پیشنهاد می شوند:

افزودن اعلان رسانی خودکار. یکپارچه سازی یک سرویس ایمیل شخص ثالث از طریق Supabase Edge Functions، به طوری که با هر تغییر وضعیت به replied در جدول messages، یا هر تغییر به approved در جدول comments، یک ایمیل خودکار برای کاربر ارسال شود. این قابلیت مستقیماً بر رضایت و بازگشت کاربران به سایت تأثیر می گذارد.

افزودن محدودسازی نرخ و کپچا. استفاده از یک سرویس کپچا سبک (مانند Cloudflare Turnstile) روی فرم های عمومی کامنت و تماس، و در صورت نیاز، تعریف یک Edge Function میانی برای اعمال محدودیت تعداد درخواست بر اساس آدرس IP یا شناسه ی نشست، پیش از رسیدن درخواست به جدول اصلی.

ارتقای ادیتور مقاله. جایگزینی <textarea> ساده ی فعلی با یک کتابخانه ی ادیتور سبک مبتنی بر React (مانند TipTap یا Lexical) که هم امکان فرمت دهی بصری (بدون نیاز به یادگیری نشانه گذاری متنی) و هم قابلیت درج تصویر مستقیم داخل متن مقاله را فراهم کند.

افزودن سیستم نقش بندی (Role-Based Access Control). گسترش جدول کاربران و سیاست های RLS برای پشتیبانی از چند سطح دسترسی (مدیر کامل، نویسنده، ناظر)، با تعریف یک ستون نقش در سطح کاربر و تطبیق سیاست های امنیتی هر جدول بر اساس این نقش، به جای تفکیک دوگانه ی فعلی میان anon و authenticated.

افزودن قابلیت های بلادرنگ (Realtime). بهره گیری از قابلیت Realtime داخلی Supabase (که بر پایه ی همان پایگاه داده و بدون نیاز به زیرساخت اضافی در دسترس است) برای به روزرسانی زنده ی داشبورد آنالیتیکس یا نمایش فوری پیام های تماس تازه، بدون نیاز به رفرش دستی صفحه توسط مدیر.

افزودن جست وجو و فیلتر پیشرفته. با رشد تعداد مقالات و پیام ها در طول زمان، افزودن قابلیت جست وجوی متنی (با بهره گیری از قابلیت جست وجوی کامل متن PostgreSQL) و فیلترهای ترکیبی (بر اساس وضعیت، تاریخ، یا نویسنده) در پنل مدیریت، تجربه ی مدیریت محتوا را در مقیاس بزرگ تر بهبود می بخشد.

منابع

۱. React Documentation. *React – The library for web and native user interfaces*. Retrieved from https://react.dev

۲. Vite Documentation. *Vite – Next Generation Frontend Tooling*. Retrieved from https://vitejs.dev

۳. Tailwind Labs. *Tailwind CSS Documentation*. Retrieved from https://tailwindcss.com/docs

۴. Remix Software. *React Router Documentation*. Retrieved from https://reactrouter.com

۵. Supabase Inc. *Supabase Documentation*. Retrieved from https://supabase.com/docs

۶. PostgREST. *PostgREST Documentation – REST API for any PostgreSQL Database*. Retrieved from https://postgrest.org

۷. PostgreSQL Global Development Group. *Row Security Policies*. PostgreSQL Documentation. Retrieved from https://www.postgresql.org/docs/current/ddl-rowsecurity.html

٨. React Hook Form. *React Hook Form Documentation*. Retrieved from https://react-hook-form.com

٩. Recharts. *Recharts – A composable charting library built on React components*. Retrieved from https://recharts.org

١٠. Fadi Khadra. *React Toastify Documentation*. Retrieved from https://fkhadra.github.io/react-toastify